Contents

Lab Preparation

1. **Install Node.js:**
    - Download and install from the official Node.js site (LTS version).
2. **Create a React + TypeScript app with Vite:**
    - Open a terminal and run:

```
npm create vite@latest react-ts-app --template react-ts
cd react-ts-app
npm install
npm run dev
```

    - Open the shown URL (usually http://localhost:5173) in your browser.
3. **Open the project in a code editor** (e.g., VS Code).

Module 1: Modern React Application Architecture

1. React as a UI library

React is a JavaScript library focused on building user interfaces using components. It doesn't decide routing, state management, or folder structure—you compose those around it.

Activity 1: Create your first component

1. **Open** src/App.tsx.
2. **Replace** its content with:

```
function App() {
  return (
    <div>
      <h1>My First React UI</h1>
      <p>React renders this UI as components.</p>
    </div>
  );
}

export default App;
```

3. **Save** and **refresh** the browser to see the UI.

Activity 2: Create a reusable Greeting component

1. **Create** src/components/Greeting.tsx:

```
function Greeting() {
  return <p>Welcome to React UI components!</p>;
}

export default Greeting;
```

2. **Use it** in App.tsx:

```
import Greeting from "./components/Greeting";

function App() {
  return (
    <div>
      <h1>My First React UI</h1>
      <Greeting />
    </div>
  );
}

export default App;
```

Activity 3: Add multiple UI sections

1. **Add** another component src/components/Header.tsx:

```
function Header() {
  return <header><h2>React UI Library Demo</h2></header>;
}

export default Header;
```

2. **Update** App.tsx:

```
import Header from "./components/Header";
import Greeting from "./components/Greeting";

function App() {
  return (
    <div>
      <Header />
      <Greeting />
    </div>
  );
}

export default App;
```

2. React.js features and benefits

Key features: components, JSX, state, props, and a virtual DOM. Benefits: reusable UI, predictable updates, and strong ecosystem.

Activity 1: Use props to customize text

1. **Update** Greeting.tsx:

```
type GreetingProps = {
  name: string;
};

function Greeting({ name }: GreetingProps) {
  return <p>Hello, {name}! Welcome to React.</p>;
}

export default Greeting;
```

2. **Pass props** in App.tsx:

```
<Greeting name="John" />
```

Activity 2: Add state with a counter

1. **Create** src/components/Counter.tsx:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

2. **Use** it in App.tsx:

```
import Counter from "./components/Counter";
// ...
<Counter />
```

Activity 3: Combine props and state

1. **Create** src/components/UserScore.tsx:

```
import { useState } from "react";

type UserScoreProps = {
  userName: string;
};

function UserScore({ userName }: UserScoreProps) {
  const [score, setScore] = useState(0);

  return (
    <div>
      <h3>{userName}'s Score: {score}</h3>
      <button onClick={() => setScore(score + 10)}>Add 10 points</button>
    </div>
  );
}

export default UserScore;
```

2. **Use** in App.tsx:

```
<UserScore userName="John" />
```

3. CRA vs Vite

Create React App (CRA) is older and slower to start/build; Vite is modern, faster, and now preferred for new React projects.

Activity 1: Inspect Vite project structure

1. **Look at** the root files: index.html, vite.config.ts, tsconfig.json, package.json.
2. **Open** src/main.tsx and see how React is mounted:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Activity 2: Compare dev experience

1. **Run** npm run dev and note:
    - Fast startup.
    - Instant hot reload when editing files.
2. **Change** text in App.tsx and see immediate updates.

Activity 3: Add a new script

1. **Open** package.json.
2. **Add** a script:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "lint": "eslint src --ext ts,tsx"
}
```

3. **Run** npm run lint (will work after you add ESLint later; for now, understand scripts are easy to extend).

4. Feature-based vs. type-based folder structure

Type-based: group by file type (components/, hooks/). Feature-based: group by feature (features/auth/, features/dashboard/). Feature-based scales better.

Activity 1: Current type-based structure

1. **Ensure** you have:
    - src/components/Header.tsx
    - src/components/Greeting.tsx
    - src/components/Counter.tsx
2. This is a simple type-based structure.

Activity 2: Create a feature folder

1. **Create** src/features/user/ folder.
2. **Move** UserScore.tsx into src/features/user/UserScore.tsx.
3. **Update import** in App.tsx:

```
import UserScore from "./features/user/UserScore";
```

Activity 3: Add another user-related component

1. **Create** src/features/user/UserProfile.tsx:

```
type UserProfileProps = {
  name: string;
};

function UserProfile({ name }: UserProfileProps) {
```

```
  return <p>User: {name}</p>;
}

export default UserProfile;
```

2. **Use** in App.tsx:

```
<UserProfile name="John" />
<UserScore userName="John" />
```

5. Colocation strategies

Colocation means keeping related files (components, hooks, services, types) close to each other inside the same feature folder.

Activity 1: Add a user hook

1. **Create** src/features/user/hooks/useUserLevel.ts:

```
import { useState } from "react";

export function useUserLevel() {
  const [level, setLevel] = useState(1);

  return { level, increaseLevel: () => setLevel(level + 1) };
}
```

2. **Use** it in UserProfile.tsx:

```
import { useUserLevel } from "./hooks/useUserLevel";

function UserProfile({ name }: UserProfileProps) {
  const { level, increaseLevel } = useUserLevel();

  return (
    <div>
      <p>User: {name}</p>
      <p>Level: {level}</p>
      <button onClick={increaseLevel}>Level Up</button>
    </div>
  );
}
```

Activity 2: Add a user service

1. **Create** src/features/user/services/userService.ts:

```
export function getWelcomeMessage(name: string) {
  return `Welcome, ${name}!`;
}
```

2. **Use** in UserScore.tsx:

```
import { getWelcomeMessage } from "./services/userService";

// inside component:
<p>{getWelcomeMessage(userName)}</p>
```

Activity 3: Add feature-specific types

1. **Create** src/features/user/types.ts:

```
export type User = {
  id: number;
  name: string;
  level: number;
};
```

2. **Use** in UserProfile.tsx (for future data):

```
import type { User } from "./types";

const sampleUser: User = { id: 1, name, level: 1 };
```

6. Avoiding oversized shared folders

Shared folders should contain truly reusable, generic items. Too many unrelated things there makes maintenance hard.

Activity 1: Create a minimal shared UI folder

1. **Create** src/shared/ui/Button.tsx:

```
type ButtonProps = {
  children: React.ReactNode;
  onClick?: () => void;
};

function Button({ children, onClick }: ButtonProps) {
  return (
    <button onClick={onClick} style={{ padding: "8px 12px" }}>
      {children}
    </button>
  );
}

export default Button;
```

2. **Use** in UserProfile.tsx:

```
import Button from "../../shared/ui/Button";

<Button onClick={increaseLevel}>Level Up</Button>
```

Activity 2: Move only truly shared utilities

1. **Create** src/shared/utils/formatScore.ts:

```
export function formatScore(score: number) {
 return `${score} pts`;
}
```

2. **Use** in UserScore.tsx:

```
import { formatScore } from "../../shared/utils/formatScore";

<h3>{userName}'s Score: {formatScore(score)}</h3>
```

Activity 3: Avoid dumping feature logic into shared

1. **Confirm** user-specific hooks and services stay in src/features/user/....
2. Only move something to shared if you can imagine multiple features using it.

7. Best practices for scalable React project structures

Use src/, feature-based folders, small shared folders, clear naming, and keep related files colocated. Avoid giant components/ or shared/ folders.

Activity 1: Organize root src

1. Ensure structure like:

```
src/
 main.tsx
 App.tsx
 components/
 features/
  user/
   hooks/
   services/
   types.ts
 shared/
  ui/
  utils/
```

Activity 2: Create another feature

1. **Create** src/features/dashboard/Dashboard.tsx:

```
function Dashboard() {
  return <h2>Dashboard Feature</h2>;
}

export default Dashboard;
```

2. **Use** in App.tsx:

```
import Dashboard from "./features/dashboard/Dashboard";

<Dashboard />
```

Activity 3: Keep files small and focused

1. If any component grows too large (many responsibilities), **split** it into smaller components in the same feature folder.
2. Example: create DashboardStats.tsx inside dashboard and use it from Dashboard.tsx.

Module 2: TypeScript-First React Development

1. Importance of TypeScript in React applications

TypeScript adds static typing to JavaScript, catching errors early and making components, props, and APIs safer and easier to maintain.

Activity 1: Strict typing in Greeting

1. Confirm GreetingProps is typed:

```
type GreetingProps = {
  name: string;
};
```

2. Try passing a number in App.tsx:

```
// @ts-expect-error
<Greeting name={123} />
```

3. See TypeScript error in your editor.

Activity 2: Enable strict mode

1. **Open** tsconfig.json.
2. Ensure "strict": true inside compilerOptions.
3. Fix any TypeScript warnings that appear.

Activity 3: Add typed configuration

1. **Create** src/config/appConfig.ts:

```
export const appConfig = {
  appName: "React TS Demo",
  version: "1.0.0",
} as const;
```

2. **Use** in Header.tsx:

```
import { appConfig } from "../config/appConfig";

<h2>{appConfig.appName}</h2>
```

2. Typing component props and event handlers (type vs interface)

You can use type or interface for props. Both work; type is often preferred for React props. Event handlers should be typed with React's event types.

Activity 1: Props with type

1. **Update** UserScore.tsx:

```
type UserScoreProps = {
  userName: string;
};
```

2. Ensure the component uses this type.

Activity 2: Props with interface

1. **Update** UserProfile.tsx:

```
interface UserProfileProps {
  name: string;
}
```

2. Confirm it still works.

Activity 3: Typed event handler

1. **Update** Counter.tsx:

```
import type { MouseEvent } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  const handleClick = (event: MouseEvent<HTMLButtonElement>) => {
    setCount(count + 1);
  };
```

```
  return (
    <button onClick={handleClick}>Count: {count}</button>
  );
}
```

2. Confirm no TypeScript errors.

3. Typing API responses and reusable components

Define types for API responses and use them in components to avoid mistakes when accessing data.

Activity 1: Simulated API response type

1. **Create** src/features/user/apiTypes.ts:

```
export type UserResponse = {
  id: number;
  name: string;
  level: number;
};
```

2. **Use** in a mock function userService.ts:

```
import type { UserResponse } from "../apiTypes";

export function fetchUser(): UserResponse {
  return { id: 1, name: "John", level: 1 };
}
```

Activity 2: Use typed data in UserProfile

1. **Update** UserProfile.tsx:

```
import { fetchUser } from "./services/userService";

const user = fetchUser();

<p>User: {user.name}</p>
<p>Level: {user.level}</p>
```

Activity 3: Typed reusable list component

1. **Create** src/shared/ui/List.tsx:

```
type ListProps<T> = {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
};

function List<T>({ items, renderItem }: ListProps<T>) {
  return <ul>{items.map(renderItem)}</ul>;
```

```
}

export default List;
```

2. **Use** in UserScore.tsx with a list of scores:

```
import List from "../../shared/ui/List";

const scores = [10, 20, 30];

<List
 items={scores}
 renderItem={(score) => <li key={score}>{score}</li>}
/>
```

4. Naming best practices

Use clear, consistent names: PascalCase for components, camelCase for functions/hooks, UPPER_SNAKE_CASE for constants, and meaningful file names.

Activity 1: Component naming

1. Ensure components are named with **PascalCase**:
    - Header.tsx → Header
    - UserProfile.tsx → UserProfile
2. Avoid generic names like MyComponent.tsx.

Activity 2: Hook and utility naming

1. Hooks: useUserLevel, useUserProfile.
2. Utilities: formatScore, getWelcomeMessage (camelCase).
3. Confirm file names match main export (e.g., formatScore.ts exports formatScore).

Activity 3: Constant naming

1. **Create** src/shared/constants/userConstants.ts:

```
export const DEFAULT_USER_LEVEL = 1;
```

2. **Use** in useUserLevel.ts:

```
import { DEFAULT_USER_LEVEL } from "../../shared/constants/userConstants";

const [level, setLevel] = useState(DEFAULT_USER_LEVEL);
```

5. Recommended naming conventions for enterprise applications

Enterprise apps benefit from strict, predictable naming: feature folders, clear suffixes (*Service, *Hook, *Types), and consistent casing.

Activity 1: Feature folder naming

1. Use **lowercase** or **kebab-case** for folders:
    - features/user
    - features/dashboard
2. Use **PascalCase** for component files:
    - UserProfile.tsx
    - DashboardStats.tsx

Activity 2: Suffixes for clarity

1. Services: userService.ts, authService.ts.
2. Hooks: useUserLevel.ts, useDashboardStats.ts.
3. Types: userTypes.ts, dashboardTypes.ts.

Activity 3: Apply conventions across project

1. Review all files and rename where needed:
    - Components: Something.tsx
    - Hooks: useSomething.ts
    - Utilities: somethingUtil.ts or descriptive names.
2. Keep imports aligned with these names to maintain clarity.

Module 3 — Managing State and Events in React

1. State management fundamentals

State stores dynamic values inside a component. When state changes, React re-renders the UI.

Activity 1 — Create a basic state component

1. In your project, create folder: src/features/state/
2. Create file: src/features/state/BasicState.tsx
3. Add:

```
import { useState } from "react";

export default function BasicState() {
 const [count, setCount] = useState(0);

 return (
  <div>
   <p>Count: {count}</p>
   <button onClick={() => setCount(count + 1)}>Increment</button>
```

```
  </div>
 );
}
```

4. Open src/App.tsx.
5. Import and render:

```
import BasicState from "./features/state/BasicState";
<BasicState />
```

Activity 2 — Add another state value

1. Open BasicState.tsx.
2. Add another state:

```
const [active, setActive] = useState(false);
```

3. Add a toggle button:

```
<button onClick={() => setActive(!active)}>
 Toggle Active
</button>
```

4. Display it:

```
<p>Active: {active ? "Yes" : "No"}</p>
```

Activity 3 — Observe re-renders

1. Open browser.
2. Click both buttons.
3. Notice UI updates immediately.

2. Local vs. derived state

Local state is stored directly. Derived state is computed from existing state and should not be stored separately.

Activity 1 — Create local values

1. Create file: src/features/state/DerivedState.tsx
2. Add:

```
import { useState } from "react";

export default function DerivedState() {
 const [a, setA] = useState(2);
 const [b, setB] = useState(3);

 const sum = a + b;

 return (
```

```
  <div>
   <p>A: {a}</p>
   <p>B: {b}</p>
   <p>Sum: {sum}</p>
  </div>
 );
}
```

3. Add to App.tsx:

```
<DerivedState />
```

Activity 2 — Add controls

1. Add buttons:

```
<button onClick={() => setA(a + 1)}>Increase A</button>
<button onClick={() => setB(b + 1)}>Increase B</button>
```

2. Observe sum updating automatically.

Activity 3 — Avoid storing derived state

1. Try adding:

```
const [sum, setSum] = useState(0);
```

2. Notice it becomes outdated.
3. Remove it and rely on const sum = a + b.

3. Controlled vs. uncontrolled components

Controlled: React stores the input value. Uncontrolled: DOM stores the value via ref.

Activity 1 — Controlled input

1. Create file: src/features/state/ControlledInput.tsx
2. Add:

```
import { useState } from "react";

export default function ControlledInput() {
 const [name, setName] = useState("");

 return (
  <div>
   <input
    value={name}
    onChange={(e) => setName(e.target.value)}
   />
   <p>Name: {name}</p>
```

```
  </div>
 );
}
```

3. Add to App.tsx.

Activity 2 — Uncontrolled input

1. Create file: src/features/state/UncontrolledInput.tsx
2. Add:

```
import { useRef } from "react";

export default function UncontrolledInput() {
 const inputRef = useRef<HTMLInputElement>(null);

 return (
  <div>
   <input ref={inputRef} />
   <button onClick={() => alert(inputRef.current?.value)}>
    Show Value
   </button>
  </div>
 );
}
```

3. Add to App.tsx.

Activity 3 — Compare

1. Type in both inputs.
2. Notice controlled input updates UI instantly.
3. Uncontrolled input only updates when button is clicked.

4. Prop drilling and lifting state up

Prop drilling: passing props through multiple layers. Lifting state: moving state to a parent so children share it.

Activity 1 — Create parent container

1. Create file: src/features/user/UserContainer.tsx
2. Add:

```
import { useState } from "react";
import UserProfile from "./UserProfile";
import UserScore from "./UserScore";

export default function UserContainer() {
 const [name, setName] = useState("John");
```

```
 return (
  <>
   <UserProfile name={name} />
   <UserScore userName={name} />
  </>
 );
}
```

3. Add to App.tsx.

Activity 2 — Add setter to children

1. Modify UserProfile.tsx:

```
type Props = { name: string; setName: (n: string) => void };
```

2. Add input:

```
<input value={name} onChange={(e) => setName(e.target.value)} />
```

Activity 3 — Observe shared updates

1. Change name in UserProfile.
2. See updated name in UserScore.

5. Event handling and binding

React uses synthetic events. Handlers receive typed event objects.

Activity 1 — Typed click event

1. Create file: src/features/events/ClickEvent.tsx
2. Add:

```
import { MouseEvent } from "react";

export default function ClickEvent() {
 const handleClick = (e: MouseEvent<HTMLButtonElement>) => {
  alert("Clicked");
 };

 return <button onClick={handleClick}>Click</button>;
 };
```

3. Add to App.tsx.

Activity 2 — Typed input event

1. Add:

```
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  console.log(e.target.value);
};
```

2. Add input:

```
<input onChange={handleChange} />
```

Activity 3 — Prevent default

1. Add form:

```
<form onSubmit={(e) => e.preventDefault()}>
  <button type="submit">Submit</button>
</form>
```

6. When to keep state local vs. lifting it

Local state: used by one component. Lifted state: shared by multiple components.

Activity 1 — Keep local

1. Open Counter.tsx.
2. Confirm count stays inside the component.

Activity 2 — Lift shared state

1. Move name from UserProfile into UserContainer.

Activity 3 — Evaluate

1. Ask: “Does another component need this value?”
2. If yes → lift.
3. If no → keep local.

7. When to use Redux Toolkit

Use Redux Toolkit when state is global, shared across many features, or complex.

Activity 1 — Install Redux Toolkit

1. Run:

```
npm install @reduxjs/toolkit react-redux
```

Activity 2 — Create store

1. Create folder: src/store/
2. Create file: src/store/store.ts
3. Add:

```
import { configureStore } from "@reduxjs/toolkit";

export const store = configureStore({
  reducer: {}
});
```

Activity 3 — Wrap App

1. Open src/main.tsx.
2. Wrap <App />:

```
import { Provider } from "react-redux";
import { store } from "./store/store";

<Provider store={store}>
  <App />
</Provider>
```

8. Common state management pitfalls

Avoid storing derived state, deeply nested objects, unnecessary lifting, and global state for simple values.

Activity 1 — Remove derived state

1. Delete:

```
const [sum, setSum] = useState(0);
```

2. Replace with:

```
const sum = a + b;
```

Activity 2 — Flatten nested state

1. Replace:

```
const [user, setUser] = useState({ profile: {}, stats: {} });
```

2. With:

```
const [name, setName] = useState("");
const [level, setLevel] = useState(1);
```

Activity 3 — Avoid global state

1. Keep simple values inside components.
2. Only use Redux for shared or complex state.

Module 4 — React Hooks and Custom Hooks

1. Core hooks

Hooks let components manage state, effects, refs, memoization, callbacks, reducers, and unique IDs.

Activity 1 — useRef

1. Create file: src/features/hooks/RefDemo.tsx
2. Add:

```
import { useRef } from "react";

export default function RefDemo() {
  const divRef = useRef<HTMLDivElement>(null);

  return <div ref={divRef}>Hello</div>;
}
```

3. Add to App.tsx.

Activity 2 — useMemo

1. Add:

```
const expensive = useMemo(() => count * 1000, [count]);
```

2. Display:

```
<p>{expensive}</p>
```

Activity 3 — useCallback

1. Add:

```
const increment = useCallback(() => setCount(c => c + 1), []);
```

2. Use:

```
<button onClick={increment}>+</button>
```

2. Managing dependencies and avoiding unnecessary effects

Effects run when dependencies change. Incorrect dependencies cause infinite loops or stale values.

Activity 1 — Correct dependency

1. Add:

```
useEffect(() => {
  console.log("count changed");
}, [count]);
```

Activity 2 — Avoid missing dependency

1. Try removing [count].
2. Notice stale logs.
3. Put [count] back.

Activity 3 — Avoid unnecessary effect

1. Replace:

```
useEffect(() => setTotal(a + b), [a, b]);
```

2. With:

```
const total = a + b;
```

3. When not to use useEffect

Avoid using useEffect for derived values, simple calculations, or synchronous logic.

Activity 1 — Remove effect

1. Delete:

```
useEffect(() => setSum(a + b), [a, b]);
```

2. Replace with:

```
const sum = a + b;
```

Activity 2 — Avoid effect for filtering

1. Replace:

```
useEffect(() => setFiltered(items.filter(...)), [items]);
```

2. With:

```
const filtered = useMemo(() => items.filter(...), [items]);
```

Activity 3 — Avoid effect for initial values

1. Replace:

```
useEffect(() => setValue(computeInitial()), []);
```

2. With:

```
const [value] = useState(() => computeInitial());
```

4. Creating and structuring custom hooks

Custom hooks encapsulate reusable logic.

Activity 1 — Create hook

1. Create file: src/features/user/hooks/useUserPoints.ts
2. Add:

```
import { useState } from "react";

export function useUserPoints() {
  const [points, setPoints] = useState(0);
  return { points, addPoints: () => setPoints(p => p + 10) };
}
```

Activity 2 — Use hook

1. Open UserProfile.tsx.
2. Add:

```
const { points, addPoints } = useUserPoints();
```

Activity 3 — Add UI

1. Add:

```
<p>Points: {points}</p>
<button onClick={addPoints}>Add</button>
```

5. Extracting reusable business logic

Move repeated logic into custom hooks.

Activity 1 — Identify repeated logic

1. Open UserProfile.tsx and UserScore.tsx.
2. Notice both use “points” logic.

Activity 2 — Create reusable hook

1. Create file: src/features/user/hooks/usePoints.ts
2. Add:

```
import { useState } from "react";

export function usePoints(initial = 0) {
  const [points, setPoints] = useState(initial);
  const add = (value: number) => setPoints(p => p + value);
  const reset = () => setPoints(initial);
  return { points, add, reset };
}
```

Activity 3 — Use in two components

1. Open UserProfile.tsx:

```
const { points, add } = usePoints();
```

2. Open UserScore.tsx:

```
const { points, add } = usePoints(10);
```

6. Common mistakes and anti-patterns

Avoid conditional hooks, unnecessary effects, derived state, and global state overuse.

Activity 1 — Fix conditional hook

1. Replace:

```
if (user) {
  const [level, setLevel] = useState(1);
}
```

2. With:

```
const [level, setLevel] = useState(1);
if (!user) return null;
```

Activity 2 — Remove unnecessary effect

1. Delete:

```
useEffect(() => setTotal(a + b), [a, b]);
```

2. Replace with:

```
const total = a + b;
```

Activity 3 — Simplify state

1. Replace:

```
const [user, setUser] = useState({ profile: {}, stats: {} });
```

2. With:

```
const [name, setName] = useState("");
const [level, setLevel] = useState(1);
```

Module 5 — Styling and Responsive UI Patterns

1. Styling approaches in React

React supports plain CSS, CSS Modules, utility-first CSS (Tailwind), and component libraries.

Activity 1 — Global CSS

1. Create folder: src/styles/.
2. Create file: src/styles/global.css.
3. Add:

```
body {
  font-family: system-ui, sans-serif;
  margin: 0;
}

.page-title {
  font-size: 24px;
  font-weight: 600;
  margin-bottom: 16px;
}
```

4. Open src/main.tsx.
5. Import:

```
import "./styles/global.css";
```

6. Open App.tsx and use:

```
<h1 className="page-title">Styling Demo</h1>
```

Activity 2 — Component-scoped CSS

1. Create folder: src/components/Button/.
2. Create file: src/components/Button/Button.tsx.
3. Add:

```
import "./Button.css";

export default function Button() {
  return <button className="btn">Click</button>;
}
```

4. Create file: src/components/Button/Button.css.
5. Add:

```
.btn {
  padding: 8px 16px;
  background: #2563eb;
  color: white;
```

```
  border-radius: 4px;
  border: none;
  cursor: pointer;
}
```

6. Import Button in App.tsx and render it.

Activity 3 — Inline styles

1. In App.tsx, add:

```
const boxStyle = {
  padding: "12px",
  backgroundColor: "#f3f4f6",
};

<div style={boxStyle}>Inline styled box</div>
```

2. Save and check the browser.

2. CSS Modules, Tailwind CSS, and component libraries

CSS Modules: scoped CSS. Tailwind: utility-first classes. Component libraries: prebuilt UI components.

Activity 1 — CSS Modules

1. Create file: src/components/Card.module.css.
2. Add:

```
.card {
  padding: 16px;
  border-radius: 8px;
  background: white;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
```

3. Create file: src/components/Card.tsx.
4. Add:

```
import styles from "./Card.module.css";

export default function Card() {
  return <div className={styles.card}>CSS Module Card</div>;
}
```

5. Import Card in App.tsx and render it.

Activity 2 — Install Tailwind CSS

1. In terminal, run:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

2. Open tailwind.config.cjs (or .js) and set:

```
export default {
 content: ["./index.html", "./src/**/*.{ts,tsx}"],
 theme: { extend: {} },
 plugins: [],
};
```

3. Open src/index.css (or main CSS file).
4. Replace content with:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

5. Ensure index.css is imported in main.tsx (Vite default).

Activity 3 — Use Tailwind classes

1. Open App.tsx.
2. Add:

```
<button className="px-4 py-2 bg-blue-600 text-white rounded">
 Tailwind Button
</button>
```

3. Save and check the browser.

3. Comparison of styling approaches

Each approach has trade-offs: global CSS is simple, CSS Modules avoid conflicts, Tailwind is fast for building UIs, libraries give ready-made components.

Activity 1 — Same button in three styles

1. **Global CSS button**
    - Add in global.css:

```
.global-btn {
 padding: 8px 16px;
 background: #16a34a;
 color: white;
 border-radius: 4px;
 border: none;
}
```

- Use in App.tsx:

```
<button className="global-btn">Global CSS</button>
```

2. **CSS Module button**
   - Create Button.module.css:

```
.btn {
  padding: 8px 16px;
  background: #dc2626;
  color: white;
  border-radius: 4px;
  border: none;
}
```

   - Create ButtonModule.tsx:

```
import styles from "./Button.module.css";

export default function ButtonModule() {
  return <button className={styles.btn}>CSS Module</button>;
}
```

3. **Tailwind button**
   - Add in App.tsx:

```
<button className="px-4 py-2 bg-indigo-600 text-white rounded">
  Tailwind
</button>
```

Activity 2 — Add a simple component library

We'll use **Chakra UI** as an example.

1. Run:

```
npm install @chakra-ui/react @emotion/react @emotion/styled framer-motion
```

2. Open main.tsx.
3. Wrap App:

```
import { ChakraProvider, Button } from "@chakra-ui/react";

ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(
  <React.StrictMode>
    <ChakraProvider>
      <App />
    </ChakraProvider>
  </React.StrictMode>
);
```

4. In App.tsx, add:

```
<Button colorScheme="teal">Chakra Button</Button>
```

Activity 3 — Visual comparison

1. Place all four buttons in App.tsx:
    - Global CSS
    - CSS Module
    - Tailwind
    - Chakra
2. Observe:
    - Where styles live (CSS file vs JSX).
    - How easy it is to change colors and spacing.

4. Tailwind CSS best practices

Use consistent spacing, colors, and extracted components. Avoid overly long class lists by using small reusable components.

Activity 1 — Create a Tailwind button component

1. Create file: src/components/TwButton.tsx.
2. Add:

```
type TwButtonProps = {
  children: React.ReactNode;
};

export default function TwButton({ children }: TwButtonProps) {
  return (
    <button className="px-4 py-2 bg-blue-600 text-white rounded hover:bg-blue-700">
      {children}
    </button>
  );
}
```

3. Use in App.tsx:

```
<TwButton>Save</TwButton>
```

Activity 2 — Use Tailwind design tokens

1. Open tailwind.config.cjs.
2. Extend theme:

```
theme: {
  extend: {
    colors: {
      brand: {
        DEFAULT: "#2563eb",
        dark: "#1d4ed8",
```

```
    },
   },
  },
},
```

3. Use in TwButton:

```
className="px-4 py-2 bg-brand text-white rounded hover:bg-brand-dark"
```

Activity 3 — Extract common layout classes

1. Create file: src/components/PageContainer.tsx.
2. Add:

```
export default function PageContainer({ children }: { children: React.ReactNode }) {
 return <div className="max-w-5xl mx-auto px-4 py-6">{children}</div>;
}
```

3. Wrap App content:

```
<PageContainer>
 {/* existing content */}
</PageContainer>
```

5. Responsive design patterns

Use flexbox, grid, and responsive Tailwind classes (sm:, md:, lg:) to adapt layouts.

Activity 1 — Responsive card grid

1. Create file: src/components/ResponsiveGrid.tsx.
2. Add:

```
export default function ResponsiveGrid() {
 return (
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
   <div className="bg-white p-4 shadow">Card 1</div>
   <div className="bg-white p-4 shadow">Card 2</div>
   <div className="bg-white p-4 shadow">Card 3</div>
  </div>
 );
}
```

3. Use in App.tsx inside PageContainer.

Activity 2 — Responsive header

1. Create file: src/components/ResponsiveHeader.tsx.
2. Add:

```
export default function ResponsiveHeader() {
 return (
  <header className="flex flex-col md:flex-row md:items-center md:justify-between gap-4 mb-6">
   <h1 className="text-2xl font-semibold">Dashboard</h1>
   <div className="flex gap-2">
    <button className="px-3 py-2 bg-gray-200 rounded">Filter</button>
    <button className="px-3 py-2 bg-blue-600 text-white rounded">New</button>
   </div>
  </header>
 );
}
```

3. Add to App.tsx above ResponsiveGrid.

Activity 3 — Test responsiveness

1. Open the app in the browser.
2. Resize the window to mobile width.
3. Observe:
    - Header stacking vertically.
    - Grid changing from 3 columns to 1.

6. Reusable layout components and dashboard design

Create layout components (header, sidebar, content) and reuse them across pages.

Activity 1 — Create a dashboard layout

1. Create file: src/layouts/DashboardLayout.tsx.
2. Add:

```
type Props = { children: React.ReactNode };

export default function DashboardLayout({ children }: Props) {
 return (
  <div className="min-h-screen bg-gray-100 flex">
   <aside className="w-64 bg-white shadow hidden md:block">
    <div className="p-4 font-semibold">Menu</div>
    <nav className="p-4 space-y-2">
     <button className="block w-full text-left">Overview</button>
     <button className="block w-full text-left">Reports</button>
    </nav>
   </aside>
   <main className="flex-1">
    <div className="max-w-5xl mx-auto px-4 py-6">{children}</div>
```

```
    </main>
   </div>
  );
}
```

3. Wrap your main content in App.tsx:

```
import DashboardLayout from "./layouts/DashboardLayout";

function App() {
  return (
   <DashboardLayout>
    <ResponsiveHeader />
    <ResponsiveGrid />
   </DashboardLayout>
  );
}
```

Activity 2 — Add reusable card component

1. Create file: src/components/DashboardCard.tsx.
2. Add:

```
type Props = {
  title: string;
  value: string;
};

export default function DashboardCard({ title, value }: Props) {
  return (
   <div className="bg-white p-4 rounded shadow">
    <p className="text-sm text-gray-500">{title}</p>
    <p className="text-xl font-semibold">{value}</p>
   </div>
  );
}
```

3. Use inside ResponsiveGrid:

```
<DashboardCard title="Users" value="1,234" />
<DashboardCard title="Revenue" value="$12,345" />
<DashboardCard title="Sessions" value="8,765" />
```

Activity 3 — Build a simple dashboard page

1. Ensure App.tsx uses:

```
<DashboardLayout>
  <ResponsiveHeader />
  <ResponsiveGrid />
</DashboardLayout>
```

2. Open the app.
3. You now have:
    - A reusable layout.
    - Responsive header.
    - Responsive cards.
    - Tailwind-based styling.

Module 6 — React Hook Form and Validation

React Hook Form fundamentals

React Hook Form (RHF) manages form state with minimal re-renders using useForm.

Activity 1: Install and basic form

1. Open terminal in your project root.
2. Run:

```
npm install react-hook-form
```

3. Create folder: src/features/forms/.
4. Create file: src/features/forms/UserFormBasic.tsx.
5. Add:

```
import { useForm } from "react-hook-form";

type FormValues = {
  name: string;
  email: string;
};

export default function UserFormBasic() {
  const { register, handleSubmit } = useForm<FormValues>();

  const onSubmit = (data: FormValues) => {
    console.log("Submitted:", data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("name")} placeholder="Name" />
      <input {...register("email")} placeholder="Email" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

6. Open src/App.tsx and render:

```
import UserFormBasic from "./features/forms/UserFormBasic";

function App() {
  return <UserFormBasic />;
}
```

7. Start dev server and submit the form; check console for data.

Activity 2: Add required validation

1. Open UserFormBasic.tsx.
2. Update inputs:

```
<input
  {...register("name", { required: "Name is required" })}
  placeholder="Name"
/>

<input
  {...register("email", { required: "Email is required" })}
  placeholder="Email"
/>
```

3. Add error display:

```
const { register, handleSubmit, formState: { errors } } =
  useForm<FormValues>();
```

4. Under each input:

```
{errors.name && <p>{errors.name.message}</p>}
{errors.email && <p>{errors.email.message}</p>}
```

5. Submit with empty fields and confirm errors appear.

Activity 3: Understand uncontrolled behavior

1. Type values in the form.
2. Submit and see data logged.
3. Notice you did not use useState for each field—RHF manages values internally.

Form registration and controlled components

RHF uses register for standard inputs and Controller for controlled components.

Activity 1: Register basic inputs

1. Confirm UserFormBasic uses:

```
<input {...register("name")} />
<input {...register("email")} />
```

2. Understand: register("name") connects the input to RHF's internal state.
3. Submit and verify name and email appear in the logged object.

Activity 2: Add a controlled select with Controller

1. Open UserFormBasic.tsx.
2. Import:

```
import { Controller } from "react-hook-form";
```

3. Extend FormValues:

```
type FormValues = {
  name: string;
  email: string;
  role: string;
};
```

4. Get control from useForm:

```
const { register, handleSubmit, control, formState: { errors } } =
  useForm<FormValues>();
```

5. Add select:

```
<Controller
  name="role"
  control={control}
  defaultValue="user"
  render={({ field }) => (
    <select {...field}>
      <option value="user">User</option>
      <option value="admin">Admin</option>
    </select>
  )}
/>
```

6. Submit and confirm role is included in logged data.

Activity 3: Mix registered and controlled fields

1. Keep name and email using register.
2. Keep role using Controller.
3. Submit and verify all three fields are present.
4. Observe that RHF can handle both styles in one form.

Managing form state and errors

RHF exposes formState for errors and status flags.

Activity 1: Display field errors

1. Ensure useForm includes:

```
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<FormValues>();
```

2. Add validation:

```
<input
  {...register("email", {
    required: "Email is required",
    pattern: {
      value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
      message: "Invalid email format",
    },
  })}
  placeholder="Email"
/>
{errors.email && <p>{errors.email.message}</p>}
```

3. Submit invalid email and confirm error message appears.

Activity 2: Show submitting state

1. Add isSubmitting:

```
const {
  register,
  handleSubmit,
  formState: { errors, isSubmitting },
} = useForm<FormValues>();
```

2. Make onSubmit async:

```
const onSubmit = async (data: FormValues) => {
 await new Promise((r) => setTimeout(r, 1000));
 console.log(data);
};
```

3. Update button:

```
<button type="submit" disabled={isSubmitting}>
 {isSubmitting ? "Submitting..." : "Submit"}
</button>
```

4. Submit and observe button disabled during submission.

Activity 3: Style invalid fields

1. Add conditional class:

```
<input
 {...register("name", { required: "Name is required" })}
 className={errors.name ? "border-red-500" : "border-gray-300"}
 placeholder="Name"
/>
```

2. Ensure Tailwind or CSS supports these classes.
3. Submit with empty name and confirm red border appears.

Default values and reset behavior

RHF supports defaultValues and reset to control initial and reset states.

Activity 1: Set default values

1. Update useForm:

```
const { register, handleSubmit, reset, formState: { errors } } =
 useForm<FormValues>({
  defaultValues: {
   name: "John",
   email: "john@example.com",
   role: "user",
  },
 });
```

2. Reload the page and confirm fields are prefilled.

Activity 2: Add a reset button

1. Add button:

```
<button type="button" onClick={() => reset()}>
 Reset to defaults
</button>
```

2. Change field values.
3. Click reset and confirm original defaults return.

Activity 3: Reset to custom values

1. Add another button:

```
<button
 type="button"
 onClick={() =>
  reset({ name: "Jane", email: "jane@example.com", role: "admin" })
 }
>
 Reset to Jane
</button>
```

2. Click and confirm fields change to Jane's data.
3. Submit and verify new values in console.

Create vs. edit form patterns

Create forms start empty; edit forms load existing data.

Activity 1: Create form with empty defaults

1. Create src/features/forms/UserFormCreate.tsx.
2. Use:

```
const { register, handleSubmit, formState: { errors } } =
 useForm<FormValues>({
  defaultValues: {
   name: "",
   email: "",
   role: "user",
  },
 });
```

3. Render inputs and submit handler similar to UserFormBasic.
4. Use this component for "Create User" page.

Activity 2: Edit form with existing data

1. Simulate existing user:

```
const existingUser: FormValues = {
  name: "Existing User",
  email: "existing@example.com",
  role: "admin",
};
```

2. In UserFormEdit.tsx, call:

```
const { register, handleSubmit, reset, formState: { errors } } =
  useForm<FormValues>({
    defaultValues: existingUser,
  });
```

3. Render form and confirm fields show existing data.

Activity 3: Switch between create and edit modes

1. In a parent component, add:

```
const [mode, setMode] = useState<"create" | "edit">("create");
const existingUser = { name: "Existing User", email: "existing@example.com", role: "admin" };
```

2. Use useForm with reset:

```
const { register, handleSubmit, reset, formState: { errors } } =
  useForm<FormValues>();
```

3. When user clicks “Edit”, call:

```
reset(existingUser);
setMode("edit");
```

4. When user clicks “Create”, call:

```
reset({ name: "", email: "", role: "user" });
setMode("create");
```

5. Confirm form switches between modes correctly.

Schema validation using Zod

Zod defines validation rules; RHF uses them via zodResolver.

Activity 1: Install Zod and resolver

1. Run:

```
npm install zod @hookform/resolvers
```

2. Create folder: src/schemas/.
3. Create file: src/schemas/userSchema.ts.
4. Add:

```
import { z } from "zod";

export const userSchema = z.object({
  name: z.string().min(2, "Name must be at least 2 characters"),
  email: z.string().email("Invalid email"),
  role: z.enum(["user", "admin"]),
});

export type UserFormValues = z.infer<typeof userSchema>;
```

Activity 2: Connect Zod to RHF

1. Open UserFormBasic.tsx.
2. Import:

```
import { zodResolver } from "@hookform/resolvers/zod";
import { userSchema, UserFormValues } from "../../schemas/userSchema";
```

3. Update useForm:

```
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<UserFormValues>({
  resolver: zodResolver(userSchema),
});
```

4. Remove inline required and pattern rules from register.
5. Submit invalid data and confirm Zod messages appear.

Activity 3: Display schema-driven errors

1. Under each field:

```
{errors.name && <p>{errors.name.message}</p>}
{errors.email && <p>{errors.email.message}</p>}
{errors.role && <p>{errors.role.message}</p>}
```

2. Try short name or invalid email.
3. Confirm errors match Zod schema messages.

Reusable validation schemas

Schemas can be reused across forms and services.

Activity 1: Share schema across forms

1. Ensure userSchema.ts stays in src/schemas/.
2. Create src/features/forms/UserFormCompact.tsx.
3. Import:

```
import { userSchema, UserFormValues } from "../../schemas/userSchema";
import { zodResolver } from "@hookform/resolvers/zod";
```

4. Use:

```
const { register, handleSubmit, formState: { errors } } =
 useForm<UserFormValues>({
  resolver: zodResolver(userSchema),
 });
```

5. Render a smaller version of the form and confirm validation works identically.

Activity 2: Use inferred types in API service

1. Open src/services/userService.ts.
2. Import:

```
import type { UserFormValues } from "../schemas/userSchema";
```

3. Define function:

```
export async function saveUser(payload: UserFormValues) {
 // call API here
}
```

4. In form submit handler, call saveUser(data) and enjoy type safety.

Activity 3: Extend schema for another context

1. In userSchema.ts, create:

```
export const userWithIdSchema = userSchema.extend({
 id: z.number(),
});

export type UserWithId = z.infer<typeof userWithIdSchema>;
```

2. Use UserWithId in components that need an id.
3. Keep base userSchema for forms that don't require id.

Best practices and common mistakes

Use schema as single source of truth, avoid mixing validation styles, and keep forms maintainable.

Activity 1: Remove duplicated validation rules

1. In your forms, remove inline required and pattern rules where Zod already defines them.
2. Keep only:

```
resolver: zodResolver(userSchema)
```

3. Confirm errors still show correctly.

Activity 2: Ensure every validated field shows errors

1. For each field in userSchema, add an error display in the form.
2. Example:

```
{errors.name && <p>{errors.name.message}</p>}
```

3. Submit invalid data and verify no field fails silently.

Activity 3: Avoid huge, monolithic forms

1. If a form has many fields, split it into smaller components (e.g., UserDetailsForm, UserSettingsForm).
2. Optionally use FormProvider from RHF to share form context across components.
3. Keep each form component focused on a small subset of fields to improve readability and maintainability.

Module 7 — Working with APIs and Async

Data HTTP requests (Fetch API vs. Axios)

Activity 1: Fetch API request

1. Create folder: src/features/api/.
2. Create file: src/features/api/FetchPosts.tsx.
3. Add:

```
import { useEffect, useState } from "react";

type Post = { id: number; title: string };

export default function FetchPosts() {
 const [posts, setPosts] = useState<Post[]>([]);

 useEffect(() => {
  async function load() {
   const res = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=3");
   const data = await res.json();
   setPosts(data);
```

```
  }
  load();
 }, []);

 return (
  <ul>
   {posts.map((p) => (
    <li key={p.id}>{p.title}</li>
   ))}
  </ul>
 );
}
```

4. Open App.tsx and render <FetchPosts />.
5. Save and check the browser.

Activity 2: Install and use Axios

1. Run:

```
npm install axios
```

2. Create file: src/features/api/AxiosPosts.tsx.
3. Add:

```
import axios from "axios";
import { useEffect, useState } from "react";

type Post = { id: number; title: string };

export default function AxiosPosts() {
 const [posts, setPosts] = useState<Post[]>([]);

 useEffect(() => {
  async function load() {
   const res = await axios.get<Post[]>(
    "https://jsonplaceholder.typicode.com/posts?_limit=3"
   );
   setPosts(res.data);
  }
  load();
 }, []);

 return (
  <ul>
   {posts.map((p) => (
    <li key={p.id}>{p.title}</li>
   ))}
```

```
  </ul>
 );
}
```

4. Render <AxiosPosts /> in App.tsx.

Activity 3: Compare Fetch vs Axios

1. Open both components side by side.
2. Observe:
    - Fetch requires res.json().
    - Axios returns res.data directly.
3. Decide which style feels clearer for your project.

API service layer design

Activity 1: Create a service layer

1. Create folder: src/services/.
2. Create file: src/services/postService.ts.
3. Add:

```
import axios from "axios";

export type Post = { id: number; title: string };

export async function fetchPosts(limit = 3): Promise<Post[]> {
 const res = await axios.get<Post[]>(
  `https://jsonplaceholder.typicode.com/posts?_limit=${limit}`
 );
 return res.data;
}
```

4. Save the file.

Activity 2: Use service in a component

1. Create file: src/features/api/PostList.tsx.
2. Add:

```
import { useEffect, useState } from "react";
import { fetchPosts, Post } from "../../services/postService";

export default function PostList() {
 const [posts, setPosts] = useState<Post[]>([]);

 useEffect(() => {
  fetchPosts().then(setPosts);
 }, []);
```

```
  return (
    <ul>
      {posts.map((p) => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  );
}
```

3. Render <PostList /> in App.tsx.

Activity 3: Create Axios instance

1. Open postService.ts.
2. Replace import with:

```
import axios from "axios";

const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
});
```

3. Update function:

```
export async function fetchPosts(limit = 3): Promise<Post[]> {
  const res = await api.get<Post[]>(`/posts?_limit=${limit}`);
  return res.data;
}
```

4. Save and confirm it still works.

Typing request/response models

Activity 1: Define response type

1. In postService.ts, ensure:

```
export type Post = { id: number; title: string };
```

2. Confirm components use Post[].

Activity 2: Define request type

1. Add:

```
export type CreatePostRequest = {
  title: string;
  body: string;
  userId: number;
};
```

2. Add function:

```
export async function createPost(payload: CreatePostRequest): Promise<Post> {
 const res = await api.post<Post>("/posts", payload);
 return res.data;
}
```

Activity 3: Use types in a form

1. Create file: src/features/api/CreatePostForm.tsx.
2. Add:

```
import { useForm } from "react-hook-form";
import { createPost, CreatePostRequest } from "../../services/postService";

export default function CreatePostForm() {
 const { register, handleSubmit } = useForm<CreatePostRequest>();

 const onSubmit = async (data: CreatePostRequest) => {
  const result = await createPost(data);
  console.log("Created:", result);
 };

 return (
  <form onSubmit={handleSubmit(onSubmit)}>
   <input {...register("title")} placeholder="Title" />
   <input {...register("body")} placeholder="Body" />
   <input {...register("userId", { valueAsNumber: true })} placeholder="User ID" />
   <button type="submit">Create</button>
  </form>
 );
}
```

3. Render <CreatePostForm /> in App.tsx.

Handling loading, error, empty, and success states

Activity 1: Add loading state

1. Open PostList.tsx.
2. Add:

```
const [loading, setLoading] = useState(true);
```

3. Update effect:

```
useEffect(() => {
  fetchPosts()
    .then(setPosts)
    .finally(() => setLoading(false));
}, []);
```

4. Add:

```
if (loading) return <p>Loading...</p>;
```

Activity 2: Add error state

1. Add:

```
const [error, setError] = useState<string | null>(null);
```

2. Update effect:

```
fetchPosts()
  .then(setPosts)
  .catch(() => setError("Failed to load posts"))
  .finally(() => setLoading(false));
```

3. Add:

```
if (error) return <p>{error}</p>;
```

Activity 3: Add empty state

1. After loading and error checks, add:

```
if (posts.length === 0) return <p>No posts found.</p>;
```

2. Save and test.

API integration best practices

Activity 1: Move all API calls to services

1. Search your project for fetch(or axios.get(inside components.
2. Move each call into a service file.
3. Replace component logic with service function calls.

Activity 2: Use consistent state pattern

1. For every API component, include:

```
const [loading, setLoading] = useState(true);
const [error, setError] = useState<string | null>(null);
const [data, setData] = useState<T | null>(null);
```

2. Follow the same loading → error → empty → success structure.

Activity 3: Avoid inline URLs

1. Open all service files.
2. Ensure URLs use:

```
api.get("/posts")
```

3. Remove hardcoded URLs from components.

Avoiding duplicated logic

Activity 1: Create reusable data-fetching hook

1. Create file: src/hooks/usePosts.ts.
2. Add:

```
import { useEffect, useState } from "react";
import { fetchPosts, Post } from "../services/postService";

export function usePosts() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetchPosts()
      .then(setPosts)
      .catch(() => setError("Failed to load posts"))
      .finally(() => setLoading(false));
  }, []);

  return { posts, loading, error };
}
```

Activity 2: Use hook in multiple components

1. Open PostList.tsx.
2. Replace state and effect with:

```
const { posts, loading, error } = usePosts();
```

3. Create PostSummary.tsx and use the same hook.
4. Confirm both components share logic.

Activity 3: Centralize error messages

1. Create file: src/constants/errors.ts.
2. Add:

```
export const GENERIC_ERROR = "Something went wrong. Please try again.";
```

3. Update usePosts:

```
.catch(() => setError(GENERIC_ERROR))
```

Environment variables and security considerations

Activity 1: Create environment variable

1. Create .env at project root.
2. Add:

```
VITE_API_BASE_URL=https://jsonplaceholder.typicode.com
```

3. Restart dev server.

Activity 2: Use env variable in Axios instance

1. Open postService.ts.
2. Update:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
});
```

3. Save and confirm API still works.

Activity 3: Understand security

1. Open .env and note that all VITE_ variables are exposed to the browser.
2. Never store secrets (API keys, tokens) in frontend env files.
3. Use a backend for sensitive operations.

Error handling strategies

Activity 1: Map technical errors to user-friendly messages

1. Open usePosts.ts.
2. Update catch:

```
.catch((err) => {
  console.error(err);
  setError("Unable to load posts right now.");
})
```

3. Save and test.

Activity 2: Add retry button

1. In usePosts, add:

```
function reload() {
  setLoading(true);
  setError(null);
  fetchPosts()
    .then(setPosts)
    .catch(() => setError("Unable to load posts."))
    .finally(() => setLoading(false));
}
```

2. Return reload:

```
return { posts, loading, error, reload };
```

3. In PostList.tsx, add:

```
{error && <button onClick={reload}>Retry</button>}
```

Activity 3: Combine form and API errors

1. In CreatePostForm.tsx, add:

```
const [serverError, setServerError] = useState<string | null>(null);
```

2. Update submit:

```
try {
  const result = await createPost(data);
  console.log(result);
} catch {
  setServerError("Failed to create post.");
}
```

3. Display:

```
{serverError && <p>{serverError}</p>}
```

Module 8 — Managing Data with Redux Toolkit

Redux fundamentals

Redux manages global state using a predictable flow: **dispatch → reducer → new state → UI updates**. Redux Toolkit simplifies Redux by reducing boilerplate and enforcing best practices.

Activity 1: Install Redux Toolkit and React-Redux

1. Open terminal in your project root.
2. Run:

```
npm install @reduxjs/toolkit react-redux
```

3. Confirm installation completes without errors.

Activity 2: Create the Redux store

1. Create folder: src/store/.
2. Create file: src/store/store.ts.
3. Add:

```
import { configureStore } from "@reduxjs/toolkit";

export const store = configureStore({
  reducer: {},
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

4. Open src/main.tsx.
5. Wrap <App />:

```
import { Provider } from "react-redux";
import { store } from "./store/store";

<Provider store={store}>
  <App />
</Provider>
```

Activity 3: Verify Redux DevTools connection

1. Open your browser.
2. Install the **Redux DevTools Extension** if you don't have it.
3. Reload your app.
4. Confirm that the store appears in DevTools.

When to use (and not use) Redux Toolkit

Use RTK when state is **global**, **shared**, or **complex**. Do NOT use RTK for simple UI state like modal toggles or form fields.

Activity 1: Identify global state

1. List states in your app (user, posts, settings, UI flags).
2. Mark which ones are used by multiple components.
3. Choose one (e.g., “user”) to move into Redux.

Activity 2: Identify local-only state

1. Open a component like Counter.tsx.
2. Confirm its state is only used inside that component.
3. Decide **not** to move it to Redux.

Activity 3: Create a “global vs local” checklist

1. Create file: docs/state-decisions.md.
2. Add:
    - Shared across features → Redux
    - Needed by many components → Redux
    - Temporary UI state → local
    - Form fields → React Hook Form
3. Use this checklist for future decisions.

Store setup, slices, reducers, and actions

Redux Toolkit organizes logic into **slices**, each owning its domain.

Activity 1: Create a slice

1. Create folder: src/features/user/.
2. Create file: src/features/user/userSlice.ts.
3. Add:

```
import { createSlice, PayloadAction } from "@reduxjs/toolkit";

type UserState = {
  name: string | null;
};

const initialState: UserState = {
  name: null,
};

const userSlice = createSlice({
  name: "user",
  initialState,
  reducers: {
    setName(state, action: PayloadAction<string>) {
      state.name = action.payload;
    },
```

```
  clearName(state) {
    state.name = null;
  },
 },
});

export const { setName, clearName } = userSlice.actions;
export default userSlice.reducer;
```

Activity 2: Add slice to store

1. Open src/store/store.ts.
2. Update:

```
import userReducer from "../features/user/userSlice";

export const store = configureStore({
 reducer: {
  user: userReducer,
 },
});
```

Activity 3: Dispatch actions from a component

1. Create file: src/features/user/UserNameSetter.tsx.
2. Add:

```
import { useDispatch } from "react-redux";
import { setName } from "./userSlice";
import type { AppDispatch } from "../../store/store";

export default function UserNameSetter() {
 const dispatch = useDispatch<AppDispatch>();

 return (
  <button onClick={() => dispatch(setName("john"))}>
   Set User Name
  </button>
 );
}
```

3. Render <UserNameSetter /> in App.tsx.
4. Check Redux DevTools to see the action.

Selectors and component integration

Selectors encapsulate derived logic and prevent unnecessary re-renders.

Activity 1: Create a selector

1. Create file: src/features/user/userSelectors.ts.
2. Add:

```
import type { RootState } from "../../store/store";

export const selectUserName = (state: RootState) => state.user.name;
```

Activity 2: Use selector in a component

1. Create file: src/features/user/UserNameDisplay.tsx.
2. Add:

```
import { useSelector } from "react-redux";
import { selectUserName } from "./userSelectors";

export default function UserNameDisplay() {
  const name = useSelector(selectUserName);
  return <p>User: {name ?? "None"}</p>;
}
```

3. Render <UserNameDisplay /> in App.tsx.

Activity 3: Confirm selector behavior

1. Click the button in UserNameSetter.
2. Watch UserNameDisplay update.
3. Check Redux DevTools to confirm state changes.

Redux DevTools

Redux DevTools helps inspect actions, state, and performance.

Activity 1: Inspect actions

1. Open DevTools → Redux tab.
2. Click “Actions”.
3. Trigger setName and watch it appear.

Activity 2: Inspect state changes

1. Click “State”.
2. Expand user.
3. Confirm name updates after dispatch.

Activity 3: Time-travel debugging

1. Trigger several actions.
2. Use DevTools “Jump” buttons to rewind state.
3. Observe UI updating accordingly.

UI and shared state management

Use Redux for shared UI state (e.g., theme, sidebar visibility), not for local UI toggles.

Activity 1: Create UI slice

1. Create file: src/features/ui/uiSlice.ts.
2. Add:

```
import { createSlice } from "@reduxjs/toolkit";

const uiSlice = createSlice({
  name: "ui",
  initialState: { sidebarOpen: false },
  reducers: {
    toggleSidebar(state) {
      state.sidebarOpen = !state.sidebarOpen;
    },
  },
});

export const { toggleSidebar } = uiSlice.actions;
export default uiSlice.reducer;
```

Activity 2: Add UI slice to store

1. Open store.ts.
2. Add:

```
import uiReducer from "../features/ui/uiSlice";

reducer: {
  user: userReducer,
  ui: uiReducer,
}
```

Activity 3: Use UI slice in a component

1. Create file: src/features/ui/SidebarToggle.tsx.
2. Add:

```
import { useDispatch, useSelector } from "react-redux";
import { toggleSidebar } from "./uiSlice";
import type { RootState, AppDispatch } from "../../store/store";

export default function SidebarToggle() {
  const dispatch = useDispatch<AppDispatch>();
  const open = useSelector((state: RootState) => state.ui.sidebarOpen);

  return (
    <>
```

```
    <button onClick={() => dispatch(toggleSidebar())}>
      Toggle Sidebar
    </button>
    <p>Sidebar is {open ? "Open" : "Closed"}</p>
   </>
  );
}
```

3. Render <SidebarToggle /> in App.tsx.

createAsyncThunk

Async thunks handle API calls and update state based on lifecycle actions.

Activity 1: Create async thunk

1. Open userSlice.ts.
2. Add:

```
import { createAsyncThunk } from "@reduxjs/toolkit";
import axios from "axios";

export const fetchUser = createAsyncThunk(
  "user/fetchUser",
  async () => {
    const res = await axios.get("https://jsonplaceholder.typicode.com/users/1");
    return res.data;
  }
);
```

Activity 2: Handle thunk states

1. In userSlice, add:

```
extraReducers: (builder) => {
  builder
    .addCase(fetchUser.pending, (state) => {
      state.name = "Loading...";
    })
    .addCase(fetchUser.fulfilled, (state, action) => {
      state.name = action.payload.name;
    })
    .addCase(fetchUser.rejected, (state) => {
      state.name = "Error loading user";
    });
}
```

2. Save file.

Activity 3: Dispatch thunk from component

1. Create file: src/features/user/UserLoader.tsx.
2. Add:

```
import { useDispatch } from "react-redux";
import { fetchUser } from "./userSlice";
import type { AppDispatch } from "../../store/store";

export default function UserLoader() {
 const dispatch = useDispatch<AppDispatch>();

 return (
  <button onClick={() => dispatch(fetchUser())}>
   Load User
  </button>
 );
}
```

3. Render <UserLoader /> in App.tsx.
4. Click button and watch state update.

Common anti-patterns

RTK best practices warn against several common mistakes:

- Putting UI-only state in Redux
- Storing derived values
- Writing async logic inside reducers
- Subscribing to entire slices instead of selectors

Activity 1: Remove UI-only state from Redux

1. Open uiSlice.ts.
2. Identify any fields like:

```
modalOpen: boolean
dropdownVisible: boolean
```

3. Move them into local component state instead.

Activity 2: Remove derived state

1. If a slice contains:

```
totalPrice: number
```

2. Delete it.
3. Compute derived values using selectors instead.

Activity 3: Fix over-subscribing selectors

1. Find components using:

```
useSelector((state) => state.user)
```

2. Replace with:

```
useSelector(selectUserName)
```

3. Confirm fewer re-renders in DevTools.

Module 9 — React Performance and Debugging

Performance considerations for enterprise apps

Enterprise React apps must optimize **render frequency**, **bundle size**, **network usage**, and **list rendering**. Profiling first is essential before applying optimizations.

Activity 1: Measure initial performance

1. Open your app in Chrome.
2. Open DevTools → **Performance** tab.
3. Click **Record**.
4. Interact with your app (navigate, click buttons).
5. Stop recording and inspect:
    - Long tasks
    - Slow renders
    - Layout shifts

Activity 2: Identify heavy components

1. Open DevTools → **React DevTools** → **Profiler**.
2. Click **Record**.
3. Trigger UI updates.
4. Look for components with high render times (>16ms).
5. Write down which components need optimization.

Activity 3: Create a performance checklist

1. Create file: docs/performance-checklist.md.
2. Add items:
    - Profile first
    - Check unnecessary re-renders
    - Memoize expensive components
    - Virtualize long lists
    - Code split large routes
3. Use this checklist before optimizing any feature.

React DevTools Profiler

Profiler shows which components re-render and how long they take.

Activity 1: Install React DevTools

1. Install the React DevTools browser extension.
2. Open your app.
3. Confirm the **Components** and **Profiler** tabs appear.

Activity 2: Record a profiling session

1. Open **Profiler** tab.
2. Click **Start profiling**.
3. Interact with your app.
4. Click **Stop profiling**.
5. Inspect:
    - Render duration
    - Commit times
    - Components causing slow updates

Activity 3: Identify slow commits

1. In Profiler, sort by **Render duration**.
2. Highlight components with high render cost.
3. Add them to your optimization list.

Identifying unnecessary re-renders

Most React performance issues come from unnecessary re-renders.

Activity 1: Add logging to detect re-renders

1. Open a component (e.g., UserProfile.tsx).
2. Add:

```
console.log("UserProfile rendered");
```

3. Interact with the app.
4. Observe how often the component re-renders.

Activity 2: Check prop changes

1. Add:

```
console.log("Props:", props);
```

2. Trigger parent updates.
3. See if props change unnecessarily.

Activity 3: Move state closer to where it's used

1. Identify a component that receives props it doesn't need.
2. Move state into the component that actually uses it.
3. Confirm fewer re-renders in Profiler.

Memoization strategies (React.memo, useMemo, useCallback)

Memoization prevents re-renders **only when components are expensive**. Overuse hurts performance.

Activity 1: Use React.memo on expensive component

1. Create file: src/components/ExpensiveChart.tsx.
2. Add:

```
function ExpensiveChart({ data }: { data: number[] }) {
  console.log("ExpensiveChart rendered");
  return <div>{data.join(", ")}</div>;
}

export default React.memo(ExpensiveChart);
```

3. Render it with stable props.
4. Confirm fewer re-renders.

Activity 2: Use useCallback for stable functions

1. Open a parent component.
2. Add:

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

3. Pass handleClick to a child.
4. Confirm child does not re-render unnecessarily.

Activity 3: Use useMemo for expensive calculations

1. Add:

```
const expensiveValue = useMemo(() => {
  return items.reduce((a, b) => a + b, 0);
}, [items]);
```

2. Confirm calculation runs only when items changes.

Handling large lists and virtualization

Rendering thousands of DOM nodes kills performance. Virtualization renders only visible items.

Activity 1: Install react-window

1. Run:

```
npm install react-window
```

2. Create file: src/features/list/VirtualizedList.tsx.

Activity 2: Implement virtualized list

1. Add:

```
import { FixedSizeList as List } from "react-window";

const items = Array.from({ length: 10000 }, (_, i) => `Item ${i}`);

export default function VirtualizedList() {
 return (
  <List
   height={400}
   itemCount={items.length}
   itemSize={35}
   width={300}
  >
   {({ index, style }) => (
    <div style={style}>{items[index]}</div>
   )}
  </List>
 );
}
```

2. Render <VirtualizedList /> in App.tsx.

Activity 3: Compare with non-virtualized list

1. Create NonVirtualizedList.tsx with:

```
items.map((i) => <div>{i}</div>)
```

2. Render both lists.
3. Observe lag in non-virtualized version.

Code splitting and lazy loading

Large bundles slow initial load. Code splitting reduces bundle size.

Activity 1: Create lazy-loaded component

1. Create file: src/features/heavy/HeavyComponent.tsx.
2. Add:

```
export default function HeavyComponent() {
  return <div>Heavy Component Loaded</div>;
}
```

Activity 2: Lazy load with React.lazy

1. In App.tsx, add:

```
const HeavyComponent = React.lazy(() =>
  import("./features/heavy/HeavyComponent")
);
```

2. Wrap with Suspense:

```
<Suspense fallback={<p>Loading...</p>}>
  <HeavyComponent />
</Suspense>
```

Activity 3: Measure bundle size impact

1. Run:

```
npm run build
```

2. Inspect output bundle sizes.
3. Confirm heavy component is split into a separate chunk.

Debugging API, rendering, and state issues

Activity 1: Debug API calls

1. Open DevTools → **Network** tab.
2. Trigger API request.
3. Check:
    - Status code
    - Response body
    - Timing
4. Fix incorrect URLs or payloads.

Activity 2: Debug rendering issues

1. Add console.log("Rendered") inside suspicious components.
2. Use Profiler to confirm render frequency.
3. Apply memoization or state colocation.

Activity 3: Debug state issues

1. Open Redux DevTools.
2. Inspect actions and state changes.
3. Identify incorrect reducers or stale state.

Production logging strategies

Activity 1: Create logging utility

1. Create file: src/utils/logger.ts.
2. Add:

```
export function logInfo(message: string, data?: unknown) {
 if (import.meta.env.MODE === "development") {
  console.info(message, data);
 }
}
```

Activity 2: Add error logger

1. Add:

```
export function logError(message: string, error?: unknown) {
 console.error(message, error);
}
```

Activity 3: Use logger in API service

1. Open postService.ts.
2. Add:

```
import { logError } from "../utils/logger";

try {
 const res = await api.get("/posts");
 return res.data;
} catch (err) {
 logError("Failed to fetch posts", err);
 throw err;
}
```

Common performance issues

Based on sources, the most common issues are:

- Unnecessary re-renders
- Overuse of memoization
- Large bundles blocking first paint
- Rendering thousands of DOM nodes without virtualization

Activity 1: Fix unnecessary re-renders

1. Identify components re-rendering too often.
2. Wrap expensive ones in React.memo.
3. Stabilize props with useCallback and useMemo.

Activity 2: Fix large bundle issues

1. Identify heavy components.
2. Convert them to lazy-loaded modules.
3. Rebuild and confirm smaller bundle.

Activity 3: Fix slow lists

1. Replace long .map() lists with react-window.
2. Test scrolling performance.
3. Confirm smooth scrolling.

Module 10 — Unit Testing (Introduction)

Overview of unit testing

Unit testing verifies small, isolated pieces of logic (functions, components). In React, the most common tools are:

- **Jest** → test runner
- **React Testing Library (RTL)** → tests React components the way users interact with them

You'll write tests for:

- Pure functions
- UI components
- User interactions

Sample unit tests for a component and a function (React Testing Library, Jest)

Activity 1: Install Jest + RTL

1. Open terminal in your project root.
2. Run:

```
npm install --save-dev jest @types/jest ts-jest @testing-library/react @testing-library/jest-dom
```

3. Initialize Jest config:

```
npx ts-jest config:init
```

4. Open jest.config.js.
5. Add:

```
module.exports = {
  preset: "ts-jest",
  testEnvironment: "jsdom",
};
```

6. Create folder: src/__tests__/.

Activity 2: Test a simple function

1. Create file: src/utils/add.ts.
2. Add:

```
export function add(a: number, b: number) {
  return a + b;
}
```

3. Create test file: src/__tests__/add.test.ts.
4. Add:

```
import { add } from "../utils/add";

test("adds two numbers", () => {
  expect(add(2, 3)).toBe(5);
});
```

5. Run tests:

```
npx jest
```

6. Confirm test passes.

Activity 3: Test a React component

1. Create file: src/components/Greeting.tsx.
2. Add:

```
export default function Greeting({ name }: { name: string }) {
  return <p>Hello, {name}</p>;
}
```

3. Create test file: src/__tests__/Greeting.test.tsx.
4. Add:

```
import { render, screen } from "@testing-library/react";
import "@testing-library/jest-dom";
import Greeting from "../components/Greeting";

test("renders greeting message", () => {
  render(<Greeting name="john" />);
  expect(screen.getByText("Hello, john")).toBeInTheDocument();
});
```

5. Run:

```
npx jest
```

6. Confirm the component test passes.

Overview of E2E testing and a quick example (Cypress or Playwright)

E2E (end-to-end) tests simulate real user behavior in a real browser. Two popular tools:

- **Cypress** → interactive GUI runner
- **Playwright** → fast, modern, multi-browser automation

We'll use Cypress for a quick example.

Activity 1: Install Cypress

1. Run:

```
npm install --save-dev cypress
```

2. Open Cypress:

```
npx cypress open
```

3. Cypress creates a cypress/ folder automatically.

Activity 2: Create your first E2E test

1. Inside cypress/e2e/, create file: basic.cy.ts.
2. Add:

```
describe("Home page", () => {
  it("loads the app", () => {
    cy.visit("http://localhost:5173");
    cy.contains("Hello").should("exist");
  });
});
```

3. Start your Vite dev server:

```
npm run dev
```

4. Run the test from the Cypress GUI.
5. Watch Cypress open a browser and run the test.

Activity 3: Add a simple interaction test

1. Create a button in App.tsx:

```
<button onClick={() => alert("Clicked!")}>Click Me</button>
```

2. Update basic.cy.ts:

```
it("clicks the button", () => {
  cy.visit("http://localhost:5173");
  cy.contains("Click Me").click();
});
```

3. Run the test again.
4. Confirm Cypress clicks the button.

Module 11 — React vs. Next.js

React as a UI library vs. Next.js as a full-stack framework

React is a **UI library** that handles rendering and component logic. Next.js is a **full-stack framework** built on React that adds routing, server rendering, static generation, API routes, and server components.

Activity 1: Create a simple React page (Vite)

1. Open terminal.
2. Create a React app:

```
npm create vite@latest react-spa -- --template react-ts
```

3. Enter project:

```
cd react-spa
npm install
npm run dev
```

4. Open src/App.tsx and add:

```
export default function App() {
  return <h1>React SPA Page</h1>;
}
```

5. Visit the page in the browser.

Activity 2: Create a simple Next.js page

1. Open terminal.
2. Run:

```
npx create-next-app@latest next-app --ts
```

3. Enter project:

```
cd next-app
npm run dev
```

4. Open app/page.tsx and add:

```
export default function Page() {
  return <h1>Next.js Page</h1>;
}
```

5. Visit the page in the browser.

Activity 3: Compare both projects

1. Observe React SPA has:
    - No routing built-in
    - No server rendering
    - No backend API routes
2. Observe Next.js has:
    - File-based routing
    - Server rendering
    - API routes
    - Server components (App Router)

SPA vs. Next.js applications

React SPA = **client-side rendering (CSR)**.
Next.js = **hybrid rendering** (CSR + SSR + SSG + RSC).

Activity 1: Inspect React SPA rendering

1. Open React SPA in browser.
2. Open DevTools → **Network**.
3. Reload page.
4. Observe:
    - HTML is mostly empty
    - JS bundle downloads
    - UI appears only after JS executes

Activity 2: Inspect Next.js SSR rendering

1. Open Next.js app.
2. Open DevTools → **Network**.
3. Reload page.
4. Observe:
    - HTML contains actual content
    - Page appears before JS loads
    - Faster first paint (SSR)

Activity 3: Compare SEO behavior

1. Right-click → "View Page Source" in React SPA.
2. Notice minimal HTML → SEO weak.
3. Do the same in Next.js.
4. Notice full HTML → SEO strong.

Client-side, server-side, and static rendering

React SPA = CSR only. Next.js supports CSR, SSR, SSG, ISR, and RSC.

Activity 1: Create a CSR component in Next.js

1. In next-app/app/, create file: client-example.tsx.
2. Add:

```
"use client";

export default function ClientExample() {
  return <p>This is a client component.</p>;
}
```

3. Import it in page.tsx.

Activity 2: Create an SSR component

1. In page.tsx, add:

```
export default async function Page() {
  const data = await fetch("https://jsonplaceholder.typicode.com/posts/1")
    .then((r) => r.json());

  return <p>{data.title}</p>;
}
```

2. Reload page and observe server-rendered content.

Activity 3: Create an SSG page

1. In next-app/app/, create folder: static/.
2. Create file: static/page.tsx.
3. Add:

```
export const revalidate = 3600; // ISR

export default function StaticPage() {
  return <h1>Static Page</h1>;
}
```

4. Visit /static and observe instant load.

When to use React only vs. Next.js

- Use **React SPA** for dashboards, internal tools, authenticated apps, and highly interactive client-side apps.
- Use **Next.js** for SEO, marketing sites, blogs, storefronts, and anything public-facing.

Activity 1: Evaluate your project needs

1. Create file: docs/project-needs.md.
2. Add questions:
    - Do I need SEO?
    - Do I need server rendering?
    - Is the app behind authentication?
    - Do I need API routes?
3. Answer each for your project.

Activity 2: Build a simple dashboard in React SPA

1. Open React SPA.
2. Add:

```
export default function App() {
  return <h1>Dashboard (React SPA)</h1>;
}
```

3. Note: SEO irrelevant; SPA is ideal.

Activity 3: Build a simple marketing page in Next.js

1. Open Next.js app.
2. Add:

```
export default function Page() {
  return <h1>Marketing Page (Next.js)</h1>;
}
```

3. Note: SEO matters; Next.js is ideal.

Common use cases

- **React SPA** → dashboards, admin panels, internal tools, long-session apps, complex client-side interactions.
- **Next.js** → SEO pages, storefronts, blogs, documentation, landing pages, hybrid apps.

Activity 1: Build an internal tool prototype in React

1. In React SPA, create:

```
export default function App() {
  return <h1>Internal Tool</h1>;
}
```

2. Add a simple form or table.
3. Observe fast dev iteration (Vite).

Activity 2: Build a blog page in Next.js

1. In Next.js, create app/blog/page.tsx.
2. Add:

```
export default function Blog() {
  return <h1>Blog Page (SEO Ready)</h1>;
}
```

3. View page source to confirm SSR.

Activity 3: Compare deployment requirements

1. React SPA → run:

```
npm run build
```

Deploy dist/ folder to any static host (Netlify, Vercel, S3).

2. Next.js → run:

```
npm run build
```

Requires Node.js server or edge runtime for SSR.

3. Note differences in hosting complexity.

Module 12 — Next.js App Router Fundamentals

App Router overview

Next.js App Router uses **folder structure as routing**, where each folder is a route segment and special files (page.tsx, layout.tsx, loading.tsx, error.tsx, not-found.tsx, route.ts) define behavior. This is confirmed by official documentation: creating a folder + page.tsx automatically creates a route, and special files control loading, error boundaries, and 404 handling.

Activity 1: Create a new Next.js App Router project

1. Open terminal.
2. Run:

```
npx create-next-app@latest next-app --ts
```

3. Enter project:

```
cd next-app
npm run dev
```

4. Open browser → visit http://localhost:3000.
5. Confirm the default App Router structure exists (app/ folder).

Activity 2: Inspect the app directory

1. Open the project in VS Code.
2. Expand the app/ folder.
3. Confirm presence of:
    - app/page.tsx → homepage
    - app/layout.tsx → root layout
4. Note: These files define the root route and layout.

Activity 3: Create a new route

1. Inside app/, create folder: about/.
2. Inside about/, create file: page.tsx.
3. Add:

```
export default function AboutPage() {
  return <h1>About Page</h1>;
}
```

4. Visit /about in browser.

app/ directory structure

The app/ directory defines routing through nested folders. Each folder = URL segment.

Activity 1: Create nested folders

1. Inside app/, create:

```
dashboard/
  settings/
```

2. Inside settings/, create page.tsx.
3. Add:

```
export default function SettingsPage() {
  return <h1>Dashboard Settings</h1>;
}
```

4. Visit /dashboard/settings.

Activity 2: Add a segment layout

1. Inside app/dashboard/, create layout.tsx.
2. Add:

```
export default function DashboardLayout({ children }: { children: React.ReactNode }) {
 return (
  <section>
   <nav>Dashboard Navigation</nav>
   {children}
  </section>
 );
}
```

3. Visit /dashboard/settings and confirm the layout wraps the page.

Activity 3: Add a root layout

1. Open app/layout.tsx.
2. Ensure it contains:

```
export default function RootLayout({ children }: { children: React.ReactNode }) {
 return (
  <html lang="en">
   <body>{children}</body>
  </html>
 );
}
```

3. Confirm this layout wraps all routes.

File-based routing (page.tsx, layout.tsx, etc.)

Next.js App Router uses special files to define route behavior.

Activity 1: Add loading.tsx

1. Inside app/dashboard/, create loading.tsx.
2. Add:

```
export default function Loading() {
 return <p>Loading dashboard...</p>;
}
```

3. Trigger a slow fetch in dashboard/page.tsx to see the loading UI.

Activity 2: Add error.tsx

1. Inside app/dashboard/, create error.tsx.
2. Add:

```
"use client";

export default function Error({ error, reset }: { error: Error; reset: () => void }) {
 return (
  <div>
   <p>Error: {error.message}</p>
   <button onClick={reset}>Retry</button>
  </div>
 );
}
```

3. Throw an error in dashboard/page.tsx to test.

Activity 3: Add not-found.tsx

1. Inside app/dashboard/, create not-found.tsx.
2. Add:

```
export default function NotFound() {
 return <h1>Dashboard Page Not Found</h1>;
}
```

3. Visit an invalid dashboard route like /dashboard/xyz.

Dynamic and nested routes

Dynamic routes use [param] folders.

Activity 1: Create a dynamic route

1. Inside app/products/, create folder [id]/.
2. Inside [id]/, create page.tsx.
3. Add:

```
export default function ProductPage({ params }: { params: { id: string } }) {
 return <h1>Product ID: {params.id}</h1>;
}
```

4. Visit /products/123.

Activity 2: Add nested dynamic routes

1. Create:

```
app/blog/[slug]/comments/[commentId]/page.tsx
```

2. Add:

```
export default function CommentPage({ params }: { params: { slug: string; commentId: string } }) {
 return (
  <div>
   <h1>Blog: {params.slug}</h1>
   <p>Comment: {params.commentId}</p>
  </div>
 );
}
```

3. Visit /blog/post-1/comments/55.

Activity 3: Pre-render dynamic routes (optional)

1. Inside [id]/page.tsx, add:

```
export async function generateStaticParams() {
 return [{ id: "1" }, { id: "2" }];
}
```

2. Build the app:

```
npm run build
```

3. Confirm pages /products/1 and /products/2 are pre-rendered.

Route groups and metadata

Route groups organize routes without affecting the URL. Metadata defines SEO tags.

Activity 1: Create a route group

1. Inside app/, create folder:

```
(marketing)/
```

2. Inside (marketing)/about/, create page.tsx.
3. Add:

```
export default function MarketingAbout() {
 return <h1>Marketing About Page</h1>;
}
```

4. Visit /about — note URL does NOT include (marketing).

Activity 2: Add metadata to a page

1. Inside (marketing)/about/page.tsx, add:

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "About Us",
  description: "Learn about our mission and team",
};
```

2. Reload page and inspect <head> tags.

Activity 3: Add metadata to a layout

1. Inside (marketing)/layout.tsx, add:

```
export const metadata = {
  title: "Marketing Section",
};
```

2. Confirm metadata applies to all pages in the group.

Colocating route-specific components

App Router encourages placing components next to the routes they belong to.

Activity 1: Create a route-specific component

1. Inside app/dashboard/, create folder components/.
2. Create file: DashboardCard.tsx.
3. Add:

```
export default function DashboardCard({ title }: { title: string }) {
  return <div className="card">{title}</div>;
}
```

Activity 2: Use the component in the route

1. Open app/dashboard/page.tsx.
2. Add:

```
import DashboardCard from "./components/DashboardCard";

export default function DashboardPage() {
  return <DashboardCard title="Dashboard Overview" />;
}
```

Activity 3: Add another colocated component

1. Create app/dashboard/components/Stats.tsx.
2. Add:

```
export default function Stats() {
  return <p>Stats Component</p>;
}
```

3. Use it in dashboard/page.tsx.

Module 13 — Server and Client Components, Data Fetching

Server vs. Client Components

According to official Next.js documentation, **pages and layouts are Server Components by default**, meaning they run on the server, can fetch data securely, and reduce JavaScript sent to the browser. Client Components are used when you need interactivity, browser APIs, or React hooks like useState and useEffect .

Activity 1: Create a Server Component

1. Open your Next.js project.
2. Go to app/.
3. Create file: app/server-example/page.tsx.
4. Add:

```
export default async function ServerExample() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await res.json();

  return <h1>Server Component: {data.title}</h1>;
}
```

5. Visit /server-example.
6. Confirm the page loads instantly (SSR).

Activity 2: Create a Client Component

1. Inside app/server-example/, create file: LikeButton.tsx.
2. Add:

```
"use client";

import { useState } from "react";

export default function LikeButton() {
  const [likes, setLikes] = useState(0);
  return <button onClick={() => setLikes(likes + 1)}>Likes: {likes}</button>;
}
```

3. Import it in page.tsx:

```
import LikeButton from "./LikeButton";
```

4. Add <LikeButton /> below the title.
5. Visit /server-example and click the button.

Activity 3: Combine Server + Client Components

1. Confirm page.tsx (Server Component) fetches data.
2. Confirm LikeButton.tsx (Client Component) handles interactivity.
3. Observe how Server Components pass props to Client Components securely.

Use cases and benefits of Server Components

Official docs highlight that Server Components allow secure data access (API keys, tokens), reduce client-side JavaScript, and improve performance (FCP) .

Activity 1: Fetch secure data on the server

1. Create file: app/profile/page.tsx.
2. Add:

```
import { cookies } from "next/headers";

export default async function ProfilePage() {
  const cookieStore = await cookies();
  const token = cookieStore.get("AUTH_TOKEN")?.value;

  const res = await fetch("https://jsonplaceholder.typicode.com/users/1", {
    headers: { Authorization: `Bearer ${token}` },
  });

  const user = await res.json();
  return <h1>Secure Profile: {user.name}</h1>;
}
```

3. Visit /profile.
4. Confirm no token is exposed to the browser.

Activity 2: Reduce client-side JavaScript

1. Create a page with heavy data:

```
export default async function HeavyPage() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  const posts = await res.json();

  return posts.map((p: any) => <p key={p.id}>{p.title}</p>);
}
```

2. Visit /heavy.
3. Open DevTools → Network → JS.
4. Observe minimal JS downloaded.

Activity 3: Stream content progressively

1. Add artificial delay:

```
await new Promise((r) => setTimeout(r, 2000));
```

2. Add loading.tsx:

```
export default function Loading() {
  return <p>Loading content...</p>;
}
```

3. Visit the page and observe streaming behavior.

Data fetching strategies

Next.js recommends three approaches: HTTP APIs, Data Access Layer (DAL), and component-level data access, depending on project size and security needs .

Activity 1: HTTP API fetching (simple)

1. Create file: app/posts/page.tsx.
2. Add:

```
export default async function PostsPage() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
  const posts = await res.json();
  return posts.map((p: any) => <p key={p.id}>{p.title}</p>);
}
```

3. Visit /posts.

Activity 2: Create a Data Access Layer (DAL)

1. Create folder: lib/data/.
2. Create file: lib/data/getPosts.ts.
3. Add:

```
import { cache } from "react";

export const getPosts = cache(async () => {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
  return res.json();
});
```

4. Use in app/posts/page.tsx:

```
import { getPosts } from "@/lib/data/getPosts";

export default async function PostsPage() {
 const posts = await getPosts();
 return posts.map((p: any) => <p key={p.id}>{p.title}</p>);
}
```

5. Visit /posts and confirm identical behavior.

Activity 3: Component-level data access (prototype)

1. Create file: app/prototype/page.tsx.
2. Add:

```
export default async function PrototypePage() {
 const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");
 const todo = await res.json();
 return <p>{todo.title}</p>;
}
```

3. Use this for quick prototypes.

Secure API calls and environment variables

Server Components allow secure access to private environment variables (not prefixed with NEXT_PUBLIC) and prevent leaking secrets to the client .

Activity 1: Create a secure environment variable

1. Create .env.local.
2. Add:

```
API_SECRET_KEY=super-secret-key
```

3. Restart dev server.

Activity 2: Use secure env variable in a Server Component

1. Create file: app/secure/page.tsx.
2. Add:

```
export default async function SecurePage() {
 const key = process.env.API_SECRET_KEY;
 return <p>Server-only key: {key}</p>;
}
```

3. Visit /secure.
4. Confirm key appears.
5. Open DevTools → View Source → confirm key is NOT exposed.

Activity 3: Prevent accidental exposure

1. Create file: lib/data/getSecret.ts.
2. Add:

```
export async function getSecret() {
  return process.env.API_SECRET_KEY;
}
```

3. Import this function in a Client Component.
4. Observe error: private env variables cannot be accessed in Client Components.
5. Move logic back to Server Component.

Common mistakes with "use client"

Official docs warn that "use client" should be used only when necessary (state, effects, browser APIs). Overusing it breaks server optimizations and may expose secure code .

Activity 1: Identify unnecessary "use client"

1. Open any component with "use client".
2. Check if it uses:
    - useState
    - useEffect
    - browser APIs
3. If not, remove "use client".

Activity 2: Fix accidental exposure of server-only code

1. Create a Client Component:

```
"use client";
import { getSecret } from "@/lib/data/getSecret";
```

2. Observe build error.
3. Move getSecret usage into a Server Component.
4. Pass safe data as props.

Activity 3: Split components correctly

1. Create app/products/page.tsx (Server Component).
2. Fetch data on server.
3. Create ProductCard.tsx with "use client" only if it needs interactivity.
4. Pass product data from server to client component.

Module 14 — Server Actions, Route Handlers, and Mutations

Server Actions fundamentals and use cases

Server Actions let you run server-side logic directly from your React components **without creating an API route**. They run **only on the server**, can access secure environment variables, and can mutate databases safely.

Activity 1: Enable Server Actions

1. Open your Next.js project.
2. Go to app/actions/.
3. Create file: app/actions/saveMessage.ts.
4. Add:

```
"use server";

export async function saveMessage(message: string) {
  console.log("Saving message:", message);
  return { success: true };
}
```

5. This function now runs **only on the server**.

Activity 2: Call a Server Action from a Client Component

1. Create file: app/actions-demo/MessageForm.tsx.
2. Add:

```
"use client";

import { useState } from "react";
import { saveMessage } from "../actions/saveMessage";

export default function MessageForm() {
  const [msg, setMsg] = useState("");

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    await saveMessage(msg);
    alert("Message saved!");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={msg} onChange={(e) => setMsg(e.target.value)} />
      <button type="submit">Save</button>
    </form>
  );
}
```

3. Create app/actions-demo/page.tsx:

```
import MessageForm from "./MessageForm";

export default function Page() {
  return <MessageForm />;
}
```

4. Visit /actions-demo and submit the form.

Activity 3: Pass data from Server Action back to UI

1. Modify saveMessage:

```
return { success: true, saved: message };
```

2. In MessageForm, update:

```
const result = await saveMessage(msg);
alert("Saved: " + result.saved);
```

3. Submit again and confirm the returned value appears.

Form handling and validation on the server

Server Actions allow secure validation using libraries like Zod.

Activity 1: Create a Zod schema

1. Create file: app/actions/messageSchema.ts.
2. Add:

```
import { z } from "zod";

export const messageSchema = z.object({
  text: z.string().min(3, "Message must be at least 3 characters"),
});
```

Activity 2: Validate inside Server Action

1. Modify saveMessage.ts:

```
"use server";

import { messageSchema } from "./messageSchema";

export async function saveMessage(text: string) {
  const parsed = messageSchema.safeParse({ text });

  if (!parsed.success) {
    throw new Error(parsed.error.errors[0].message);
  }

  return { saved: parsed.data.text };
}
```

Activity 3: Display validation errors in the UI

1. Update MessageForm.tsx:

```
try {
  const result = await saveMessage(msg);
  alert("Saved: " + result.saved);
} catch (err: any) {
  alert("Error: " + err.message);
}
```

2. Submit a short message and confirm error appears.

Route Handlers (GET, POST, etc.)

Route Handlers are API endpoints inside the app/ directory. They replace pages/api/* in the App Router.

Activity 1: Create a GET route

1. Create folder: app/api/hello/.
2. Create file: app/api/hello/route.ts.
3. Add:

```
export function GET() {
  return Response.json({ message: "Hello from API" });
}
```

4. Visit /api/hello in browser.

Activity 2: Create a POST route

1. In the same folder, update route.ts:

```
export async function POST(req: Request) {
  const body = await req.json();
  return Response.json({ received: body });
}
```

2. Test with fetch:

```
fetch("/api/hello", {
  method: "POST",
  body: JSON.stringify({ name: "john" }),
});
```

Activity 3: Use Route Handler inside a Server Action

1. Create file: app/actions/sendToApi.ts.
2. Add:

```
"use server";

export async function sendToApi(data: string) {
  const res = await fetch("http://localhost:3000/api/hello", {
    method: "POST",
    body: JSON.stringify({ data }),
  });

  return res.json();
}
```

3. Call this action from a Client Component.

Choosing between Server Actions and API routes

Use **Server Actions** when:

- You mutate data directly from forms
- You want secure server-only logic
- You don't need a public API endpoint

Use **Route Handlers** when:

- You need a public API
- You integrate with external services
- You need REST endpoints

Activity 1: Compare both approaches

1. Create a form using Server Actions.
2. Create a form using /api/hello.
3. Observe:
    - Server Actions require no API call
    - Route Handlers require fetch()

Activity 2: Convert a Server Action to an API route

1. Move logic from saveMessage.ts into /api/message/route.ts.
2. Update Client Component to call fetch("/api/message").
3. Compare complexity.

Activity 3: Decide which to use for your project

1. Create file: docs/server-actions-vs-api.md.
2. Add criteria:
    - Need public API → Route Handler
    - Need secure server-only logic → Server Action

- Need form mutation → Server Action
- Need external integration → Route Handler

Revalidation and error handling

Next.js supports **cache revalidation** for dynamic data and built-in error boundaries.

Activity 1: Add revalidation to a fetch

1. In a Server Component:

```
const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
  next: { revalidate: 10 },
});
```

2. Visit page.
3. Data refreshes every 10 seconds.

Activity 2: Add error.tsx to a route

1. Inside app/posts/, create error.tsx.
2. Add:

```
"use client";

export default function Error({ error }: { error: Error }) {
  return <p>Error loading posts: {error.message}</p>;
}
```

3. Throw an error in page.tsx to test.

Activity 3: Add not-found.tsx

1. Inside app/posts/, create not-found.tsx.
2. Add:

```
export default function NotFound() {
  return <h1>Post not found</h1>;
}
```

3. In page.tsx, call:

```
notFound();
```

4. Visit /posts and confirm 404 behavior.

Final Capstone Project

Customer Support Ticket Management System

This app simulates a lightweight internal tool used by bank employees to:

- Create support tickets
- View all tickets
- Edit ticket details
- Delete tickets
- Validate forms
- Integrate with a real API (mockapi.io)
- Use Next.js App Router best practices
- Deploy to Vercel

PHASE 1 — Project Setup, Architecture, API Modeling, and Basic CRUD Skeleton

This phase sets the foundation for a production-ready Next.js App Router application.

1. Project Planning and Domain Definition

1.1 Choose the domain

We will build: **Customer Support Ticket Management System**

1.2 Define the core entity: Ticket

Create docs/domain.md:

```
Ticket {
  id: string
  title: string
  description: string
  priority: "low" | "medium" | "high"
  status: "open" | "in-progress" | "resolved"
  createdAt: string
}
```

1.3 Define MVP features

- Create Ticket
- View Ticket List
- Edit Ticket
- Delete Ticket
- Validate forms
- Responsive layout
- Production-ready folder structure

1.4 Define non-functional requirements

- Strong TypeScript usage
- Clean architecture
- Secure server-side logic
- Reusable components
- Minimal client-side JS
- Fast performance

2. Create the Next.js App Router Project

2.1 Create the project

```
npx create-next-app@latest support-ticket-app --ts
```

Choose:

- TypeScript: Yes
- App Router: Yes
- Tailwind: Yes
- ESLint: Yes
- Src directory: Yes

2.2 Run the project

```
cd support-ticket-app
npm run dev
```

Visit: http://localhost:3000

2.3 Clean up starter code

Replace src/app/page.tsx with:

```
export default function HomePage() {
  return <h1 className="text-2xl font-bold">Support Ticket System</h1>;
}
```

3. Create a Scalable Project Structure

Inside src/, create:

```
app/
  (dashboard)/
    layout.tsx
    page.tsx
    tickets/
      page.tsx
      loading.tsx
      error.tsx
components/
features/
  tickets/
    TicketForm.tsx
    TicketItem.tsx
    ticketService.ts
```

```
  ticketTypes.ts
lib/
 api.ts
schemas/
 ticketSchema.ts
store/
docs/
```

3.1 Create a route group for the dashboard

src/app/(dashboard)/layout.tsx:

```
export default function DashboardLayout({ children }: { children: React.ReactNode }) {
 return (
  <div className="min-h-screen flex">
   <aside className="w-64 border-r p-4">
    <h2 className="font-bold mb-4">Support Dashboard</h2>
    <nav className="space-y-2">
     <a href="/" className="block">Home</a>
     <a href="/tickets" className="block">Tickets</a>
    </nav>
   </aside>
   <main className="flex-1 p-6">{children}</main>
  </div>
 );
}
```

3.2 Move homepage into dashboard group

src/app/(dashboard)/page.tsx:

```
export default function DashboardPage() {
 return <h1 className="text-2xl font-bold">Dashboard</h1>;
}
```

4. Set Up API Using mockapi.io

4.1 Create mockapi.io project

1. Go to https://mockapi.io
2. Create a new project
3. Create a resource named **tickets**

4.2 Add fields

- title → string
- description → string
- priority → string
- status → string
- createdAt → date

4.3 Copy base URL

Example:

```
https://<your-id>.mockapi.io/api/v1
```

4.4 Add environment variable

Create .env.local:

```
NEXT_PUBLIC_API_BASE_URL=https://<your-id>.mockapi.io/api/v1
```

Restart dev server.

5. Create TypeScript Models and Zod Schemas

5.1 Create ticket types

src/features/tickets/ticketTypes.ts:

```
export type Ticket = {
  id: string;
  title: string;
  description: string;
  priority: "low" | "medium" | "high";
  status: "open" | "in-progress" | "resolved";
  createdAt: string;
};

export type CreateTicketRequest = Omit<Ticket, "id" | "createdAt">;
export type UpdateTicketRequest = Partial<CreateTicketRequest>;
```

5.2 Create Zod schema

src/schemas/ticketSchema.ts:

```
import { z } from "zod";

export const ticketSchema = z.object({
  title: z.string().min(3, "Title must be at least 3 characters"),
  description: z.string().min(5, "Description must be at least 5 characters"),
  priority: z.enum(["low", "medium", "high"]),
  status: z.enum(["open", "in-progress", "resolved"]),
});

export type TicketFormValues = z.infer<typeof ticketSchema>;
```

6. Create Data Access Layer (API Service)

6.1 Create Axios instance

Install Axios:

```
npm install axios
```

src/lib/api.ts:

```
import axios from "axios";

export const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_BASE_URL,
});
```

6.2 Create ticket service

src/features/tickets/ticketService.ts:

```
import { api } from "@/lib/api";
import type { Ticket, CreateTicketRequest, UpdateTicketRequest } from "./ticketTypes";

export async function getTickets(): Promise<Ticket[]> {
  const res = await api.get<Ticket[]>("/tickets");
  return res.data;
}

export async function createTicket(payload: CreateTicketRequest): Promise<Ticket> {
  const res = await api.post<Ticket>("/tickets", payload);
  return res.data;
}

export async function updateTicket(id: string, payload: UpdateTicketRequest): Promise<Ticket> {
  const res = await api.put<Ticket>(` /tickets/${id}` , payload);
  return res.data;
}

export async function deleteTicket(id: string): Promise<void> {
  await api.delete(` /tickets/${id}` );
}
```

7. Build Tickets List Page (Server Component)

src/app/(dashboard)/tickets/page.tsx:

```
import { getTickets } from "@/features/tickets/ticketService";
import TicketItem from "@/features/tickets/TicketItem";
import TicketForm from "@/features/tickets/TicketForm";

export default async function TicketsPage() {
  const tickets = await getTickets();
```

```
  return (
    <div className="space-y-6">
      <h1 className="text-2xl font-bold">Tickets</h1>

      <TicketForm />

      <ul className="space-y-2">
        {tickets.map((ticket) => (
          <TicketItem key={ticket.id} ticket={ticket} />
        ))}
      </ul>
    </div>
  );
}
```

7.1 Add loading state

src/app/(dashboard)/tickets/loading.tsx:

```
export default function LoadingTickets() {
  return <p>Loading tickets...</p>;
}
```

8. Create Ticket Form (Client Component + Validation)

Install RHF + Zod resolver:

```
npm install react-hook-form @hookform/resolvers zod
```

src/features/tickets/TicketForm.tsx:

```
"use client";

import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { ticketSchema, TicketFormValues } from "@/schemas/ticketSchema";
import { createTicket } from "./ticketService";
import { useRouter } from "next/navigation";

export default function TicketForm() {
  const router = useRouter();

  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<TicketFormValues>({
    resolver: zodResolver(ticketSchema),
  });
```

```
const onSubmit = async (data: TicketFormValues) => {
 await createTicket(data);
 router.refresh();
};

return (
 <form onSubmit={handleSubmit(onSubmit)} className="space-y-4 max-w-md">
  <div>
   <label className="block text-sm font-medium">Title</label>
   <input {...register("title")} className="border rounded w-full p-2" />
   {errors.title && <p className="text-red-500 text-xs">{errors.title.message}</p>}
  </div>

  <div>
   <label className="block text-sm font-medium">Description</label>
   <textarea {...register("description")} className="border rounded w-full p-2" />
   {errors.description && <p className="text-red-500 text-xs">{errors.description.message}</p>}
  </div>

  <div>
   <label className="block text-sm font-medium">Priority</label>
   <select {...register("priority")} className="border rounded w-full p-2">
    <option value="low">Low</option>
    <option value="medium">Medium</option>
    <option value="high">High</option>
   </select>
  </div>

  <div>
   <label className="block text-sm font-medium">Status</label>
   <select {...register("status")} className="border rounded w-full p-2">
    <option value="open">Open</option>
    <option value="in-progress">In Progress</option>
    <option value="resolved">Resolved</option>
   </select>
  </div>

  <button
   type="submit"
   disabled={isSubmitting}
   className="bg-blue-600 text-white px-4 py-2 rounded"
  >
   {isSubmitting ? "Saving..." : "Create Ticket"}
  </button>
 </form>
```

```
  );
}
```

9. Create TicketItem Component (Client Component)

src/features/tickets/TicketItem.tsx:

```
"use client";

import { deleteTicket } from "./ticketService";
import { useRouter } from "next/navigation";
import type { Ticket } from "./ticketTypes";

export default function TicketItem({ ticket }: { ticket: Ticket }) {
  const router = useRouter();

  const handleDelete = async () => {
    await deleteTicket(ticket.id);
    router.refresh();
  };

  return (
    <li className="border p-3 rounded flex justify-between items-start">
      <div>
        <p className="font-semibold">{ticket.title}</p>
        <p className="text-sm text-gray-600">{ticket.description}</p>
        <p className="text-xs mt-1">Priority: {ticket.priority}</p>
        <p className="text-xs">Status: {ticket.status}</p>
      </div>
      <button onClick={handleDelete} className="text-red-600 text-sm">
        Delete
      </button>
    </li>
  );
}
```

10. Phase 1 Completion Checklist

By the end of Phase 1, participants will have:

Architecture

- Scalable folder structure
- Route groups
- Shared layout

API

- mockapi.io resource
- Environment variables
- Axios instance
- CRUD service layer

TypeScript

- Strong models
- Zod schema

UI

- Tickets list page
- Create Ticket form
- Delete Ticket button
- Responsive layout

Next.js

- Server Components
- Client Components
- Loading UI

Outcome

A fully working **CRUD skeleton** of a production-ready banking-style internal tool.

PHASE 2 — Editing, Mutations, Server Actions, Error Handling, State Management, Responsive UI

1. Add Ticket Editing (Client Component + Server Action)

Editing is essential in any banking workflow. We'll add:

- Edit page
- Edit form
- Server Action for secure mutation
- Navigation from list → edit page

1.1 Create dynamic route for editing

Create folder:

```
src/app/(dashboard)/tickets/[id]/
```

Inside it, create page.tsx:

```
import { getTickets } from "@/features/tickets/ticketService";
import EditTicketForm from "@/features/tickets/EditTicketForm";

export default async function EditTicketPage({ params }: { params: { id: string } }) {
 const tickets = await getTickets();
 const ticket = tickets.find((t) => t.id === params.id);

 if (!ticket) {
  return <h1>Ticket not found</h1>;
 }

 return (
  <div className="space-y-6">
```

```
    <h1 className="text-2xl font-bold">Edit Ticket</h1>
    <EditTicketForm ticket={ticket} />
   </div>
  );
}
```

1.2 Add "Edit" button to TicketItem

Update TicketItem.tsx:

```
<a
 href={`/tickets/${ticket.id}`}
 className="text-blue-600 text-sm mr-4"
>
 Edit
</a>
```

Now each ticket has an Edit link.

1.3 Create EditTicketForm (Client Component)

src/features/tickets/EditTicketForm.tsx:

```
"use client";

import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { ticketSchema, TicketFormValues } from "@/schemas/ticketSchema";
import { updateTicket } from "./ticketService";
import { useRouter } from "next/navigation";
import type { Ticket } from "./ticketTypes";

export default function EditTicketForm({ ticket }: { ticket: Ticket }) {
 const router = useRouter();

 const {
  register,
  handleSubmit,
  formState: { errors, isSubmitting },
 } = useForm<TicketFormValues>({
  resolver: zodResolver(ticketSchema),
  defaultValues: {
   title: ticket.title,
   description: ticket.description,
   priority: ticket.priority,
   status: ticket.status,
  },
 });

 const onSubmit = async (data: TicketFormValues) => {
```

```
  await updateTicket(ticket.id, data);
  router.push("/tickets");
  router.refresh();
 };

 return (
  <form onSubmit={handleSubmit(onSubmit)} className="space-y-4 max-w-md">
   <div>
    <label className="block text-sm font-medium">Title</label>
    <input {...register("title")} className="border rounded w-full p-2" />
    {errors.title && <p className="text-red-500 text-xs">{errors.title.message}</p>}
   </div>

   <div>
    <label className="block text-sm font-medium">Description</label>
    <textarea {...register("description")} className="border rounded w-full p-2" />
    {errors.description && <p className="text-red-500 text-xs">{errors.description.message}</p>}
   </div>

   <div>
    <label className="block text-sm font-medium">Priority</label>
    <select {...register("priority")} className="border rounded w-full p-2">
     <option value="low">Low</option>
     <option value="medium">Medium</option>
     <option value="high">High</option>
    </select>
   </div>

   <div>
    <label className="block text-sm font-medium">Status</label>
    <select {...register("status")} className="border rounded w-full p-2">
     <option value="open">Open</option>
     <option value="in-progress">In Progress</option>
     <option value="resolved">Resolved</option>
    </select>
   </div>

   <button
    type="submit"
    disabled={isSubmitting}
    className="bg-green-600 text-white px-4 py-2 rounded"
   >
    {isSubmitting ? "Saving..." : "Update Ticket"}
   </button>
  </form>
```

```
  );
}
```

Now you have full CRUD: **Create, Read, Update, Delete**.

2. Introduce Server Actions for Secure Mutations

Bank programmers need secure server-side logic.

We'll convert **create** and **update** into Server Actions.

2.1 Create server actions

Create folder:

```
src/app/actions/
```

Create file: ticketActions.ts:

```
"use server";

import { api } from "@/lib/api";
import type { CreateTicketRequest, UpdateTicketRequest } from "@/features/tickets/ticketTypes";

export async function createTicketAction(payload: CreateTicketRequest) {
  const res = await api.post("/tickets", payload);
  return res.data;
}

export async function updateTicketAction(id: string, payload: UpdateTicketRequest) {
  const res = await api.put(`/tickets/${id}`, payload);
  return res.data;
}
```

2.2 Update forms to use Server Actions

Create form

Replace:

```
await createTicket(data);
```

with:

```
await createTicketAction(data);
```

Edit form

Replace:

```
await updateTicket(ticket.id, data);
```

with:

```
await updateTicketAction(ticket.id, data);
```

Now mutations run **only on the server**, not in the browser.

3. Add Route Handlers (API Endpoints)

Route Handlers are useful for:

- Internal APIs
- External integrations
- Background jobs
- Banking-style microservices

3.1 Create route handler

Create folder:

```
src/app/api/tickets/
```

Create file: route.ts:

```
import { NextResponse } from "next/server";
import { api } from "@/lib/api";

export async function GET() {
  const res = await api.get("/tickets");
  return NextResponse.json(res.data);
}

export async function POST(req: Request) {
  const body = await req.json();
  const res = await api.post("/tickets", body);
  return NextResponse.json(res.data);
}
```

3.2 Add dynamic route handler

Create folder:

```
src/app/api/tickets/[id]/
```

Create file: route.ts:

```
import { NextResponse } from "next/server";
import { api } from "@/lib/api";

export async function PUT(req: Request, { params }: { params: { id: string } }) {
  const body = await req.json();
  const res = await api.put(` /tickets/${params.id}` , body);
  return NextResponse.json(res.data);
}

export async function DELETE(req: Request, { params }: { params: { id: string } }) {
  await api.delete(` /tickets/${params.id}` );
  return NextResponse.json({ success: true });
}
```

Now your app has a **full internal API layer**.

4. Add Error Boundaries and Not Found Pages

4.1 Add error.tsx

src/app/(dashboard)/tickets/error.tsx:

```
"use client";

export default function Error({ error }: { error: Error }) {
  return (
    <div className="text-red-600">
      <h2>Error loading tickets</h2>
      <p>{error.message}</p>
    </div>
  );
}
```

4.2 Add not-found.tsx

src/app/(dashboard)/tickets/not-found.tsx:

```
export default function NotFound() {
  return <h1>Ticket not found</h1>;
}
```

5. Add State Management (Redux Toolkit or Zustand)

For simplicity and speed, we'll use **Zustand**.

Install:

```
npm install zustand
```

Create store:

src/store/ticketStore.ts:

```
import { create } from "zustand";
import type { Ticket } from "@/features/tickets/ticketTypes";

type TicketState = {
  selectedTicket: Ticket | null;
  setSelectedTicket: (ticket: Ticket | null) => void;
};

export const useTicketStore = create<TicketState>((set) => ({
  selectedTicket: null,
  setSelectedTicket: (ticket) => set({ selectedTicket: ticket }),
}));
```

Use it in TicketItem:

```
const { setSelectedTicket } = useTicketStore();
```

This allows:

- Global selection
- Future enhancements (filters, sorting, etc.)

6. Improve Responsive UI

Add Tailwind responsive classes:

- Sidebar collapses on mobile
- Buttons stack
- Forms become full-width

Update layout:

```
<div className="min-h-screen flex flex-col md:flex-row">
```

Update form:

```
className="space-y-4 w-full md:max-w-md"
```

Update list:

```
className="space-y-2 w-full"
```

Now the app works on:

- Desktop
- Tablet
- Mobile

7. Add Performance Optimizations

7.1 Memoize TicketItem

```
export default React.memo(TicketItem);
```

7.2 Use Suspense for streaming

Wrap list:

```
<Suspense fallback={<p>Loading tickets...</p>}>
  <TicketList />
</Suspense>
```

7.3 Use Server Components for heavy data

Ticket list is already a Server Component

8. Phase 2 Completion Checklist

CRUD

- Edit Ticket
- Delete Ticket
- Create Ticket
- View Tickets

Server

- Server Actions
- Route Handlers
- Secure mutations

UI

- Responsive layout
- Error boundaries
- Loading states

State

- Zustand store

Performance

- Memoization
- Suspense
- Server Components

Outcome

Your app is now **secure**, **scalable**, **responsive**, and **production-ready**.

PHASE 3 — Deployment, Security, Monitoring, Optimization, Final Delivery

1. Prepare the Application for Production

Before deploying, we ensure the app is stable, secure, and ready for real traffic.

1.1 Clean up folder structure

Your final structure should look like:

```
src/
 app/
  (dashboard)/
   layout.tsx
   page.tsx
   tickets/
    page.tsx
    loading.tsx
    error.tsx
    not-found.tsx
    [id]/
     page.tsx
  api/
   tickets/
    route.ts
   tickets/[id]/
```

```
    route.ts
 components/
 features/
  tickets/
   TicketForm.tsx
   EditTicketForm.tsx
   TicketItem.tsx
   ticketService.ts
   ticketTypes.ts
 schemas/
  ticketSchema.ts
 lib/
  api.ts
 store/
  ticketStore.ts
docs/
 domain.md
 api.md
 server-actions-vs-api.md
```

This structure mirrors enterprise-grade Next.js apps.

1.2 Add production environment variables

Create .env.production:

```
NEXT_PUBLIC_API_BASE_URL=https://<your-mockapi-project>.mockapi.io/api/v1
```

Never store secrets in NEXT_PUBLIC_*.

1.3 Add global error boundary

Create src/app/error.tsx:

```
"use client";

export default function GlobalError({ error, reset }: { error: Error; reset: () => void }) {
 return (
  <div className="p-6 text-red-600">
   <h1 className="text-xl font-bold">Something went wrong</h1>
   <p>{error.message}</p>
   <button onClick={reset} className="mt-4 bg-blue-600 text-white px-4 py-2 rounded">
    Try Again
   </button>
  </div>
 );
}
```

This catches unexpected errors across the entire app.

1.4 Add global loading UI

Create src/app/loading.tsx:

```
export default function GlobalLoading() {
  return <p className="p-6">Loading application...</p>;
}
```

2. Deployment to Vercel

Vercel is the fastest way to deploy Next.js.

2.1 Push project to GitHub

1. Initialize git:

```
git init
git add .
git commit -m "Initial production-ready version"
```

2. Create a GitHub repo.
3. Push:

```
git remote add origin <repo-url>
git push -u origin main
```

2.2 Deploy to Vercel

1. Go to https://vercel.com
2. Click **New Project**
3. Import your GitHub repo
4. Vercel auto-detects Next.js
5. Add environment variables:

```
NEXT_PUBLIC_API_BASE_URL=https://<your-mockapi>.mockapi.io/api/v1
```

6. Click **Deploy**

Your app is now live.

2.3 Validate deployment

Visit your Vercel URL:

- / → Dashboard
- /tickets → Ticket list
- /tickets/<id> → Edit page
- Create, edit, delete tickets
- Confirm API calls work
- Confirm layout is responsive

3. Production Logging & Monitoring

Bank programmers need visibility into errors and performance.

3.1 Add a logging utility

Create src/lib/logger.ts:

```
export function logInfo(message: string, data?: unknown) {
  if (process.env.NODE_ENV === "development") {
    console.info(message, data);
  }
}

export function logError(message: string, error?: unknown) {
  console.error(message, error);
}
```

Use in server actions:

```
logError("Failed to update ticket", err);
```

3.2 Add monitoring (Vercel Analytics)

1. Install:

```
npm install @vercel/analytics
```

2. Add to src/app/layout.tsx:

```
import { Analytics } from "@vercel/analytics/react";

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        {children}
        <Analytics />
      </body>
    </html>
  );
}
```

You now get:

- Page load metrics
- Web vitals
- Performance insights

4. Security Hardening

Even simple apps must follow bank-level security patterns.

4.1 Ensure all sensitive logic runs on the server

Check:

- Server Actions → safe
- Route Handlers → safe
- No secrets in Client Components
- No process.env.* in client code

4.2 Validate all incoming data

Your Server Actions already use Zod. Add validation to Route Handlers:

```
import { ticketSchema } from "@/schemas/ticketSchema";

export async function POST(req: Request) {
  const body = await req.json();
  const parsed = ticketSchema.safeParse(body);

  if (!parsed.success) {
    return NextResponse.json({ error: parsed.error }, { status: 400 });
  }

  const res = await api.post("/tickets", parsed.data);
  return NextResponse.json(res.data);
}
```

4.3 Prevent accidental client-side exposure

Search your project:

```
NEXT_PUBLIC_API_BASE_URL
```

Ensure:

- Only public values use NEXT_PUBLIC
- No private keys exist

5. Performance Optimization

Your app is already fast, but let's polish it.

5.1 Use React.memo for TicketItem

```
export default React.memo(TicketItem);
```

5.2 Use Suspense for streaming

Wrap ticket list:

```
<Suspense fallback={<p>Loading tickets...</p>}>
  <TicketList />
</Suspense>
```

5.3 Reduce client-side JS

Move logic from Client Components to Server Components when possible.

Examples:

- Ticket list → Server Component
- Ticket detail → Server Component
- Only forms remain client-side

5.4 Add caching to data layer

In getTickets:

```
export const getTickets = cache(async () => {
  const res = await api.get<Ticket[]>("/tickets");
  return res.data;
});
```

6. Final Polishing

6.1 Add metadata for SEO

src/app/(dashboard)/tickets/page.tsx:

```
export const metadata = {
  title: "Tickets",
  description: "Manage support tickets",
};
```

6.2 Add reusable UI components

Create:

- Button.tsx
- Card.tsx
- Input.tsx
- Select.tsx

Use them across forms and pages.

6.3 Add mobile-friendly navigation

Update layout:

```
<aside className="w-full md:w-64 border-r p-4">
```

7. Final Delivery Checklist

Architecture

- Clean folder structure
- Route groups
- Reusable components

TypeScript

- Strong models
- Zod schemas

CRUD

- Create

- Read
- Update
- Delete

Next.js

- Server Components
- Client Components
- Server Actions
- Route Handlers
- Error boundaries
- Loading UI

Security

- Server-side validation
- No secrets exposed
- Safe mutations

Performance

- Memoization
- Suspense
- Caching

Deployment

- Vercel deployment
- Production env vars
- Monitoring enabled

Outcome

You now have a **fully production-ready Next.js App Router application**, built with:

- Enterprise architecture
- Secure server-side logic
- Strong TypeScript
- Clean UI
- Real API integration
- Deployment and monitoring
- Performance optimization